



*Facultad
de
Ciencias*

**DESARROLLO DE UNA APLICACIÓN
PARA LA DETECCIÓN DE CONFLICTOS
EN HORARIOS ACADÉMICOS**

**(Development of a Conflict Detection Application
for Academic Timetables)**

**Trabajo de Fin de Grado
para acceder al**

GRADO EN INGENIERÍA INFORMÁTICA

Autor: Jose Martín Aramburu

Directora: Cristina Tîrnăucă

Febrero – 2026

A mis padres, familiares y amigos, por su apoyo incondicional.

A Cristina, por su dirección y confianza.

Gracias.

Resumen

Este trabajo de fin de grado tiene como objetivo el diseño y desarrollo de una aplicación web que facilite la detección automática de restricciones y conflictos en horarios académicos, a partir de los documentos PDF correspondientes a los cuadros horarios y a las fichas de las asignaturas. La propuesta busca automatizar un proceso que actualmente suele realizarse de forma manual, el cual es propenso a errores y supone un alto coste en tiempo para las instituciones.

Para ello, la aplicación permite a los usuarios cargar los documentos oficiales y extraer la información estructurada. Superando las limitaciones del reconocimiento óptico de caracteres convencional, el sistema implementa un motor de parseo nativo y reconstrucción geométrica, apoyado por algoritmos de coincidencia difusa. Una vez consolidada la información, el núcleo analítico detecta en tiempo real incidencias críticas, tales como solapamientos de clases, sobreocupación de aulas o vulneraciones en la disponibilidad y conciliación del profesorado.

Como resultado, el proyecto despliega una solución interactiva, transaccional y escalable. Al dotar a la gestión académica de un entorno de validación sistemática, el sistema no solo elimina las inconsistencias lógicas de la planificación, sino que optimiza la eficiencia administrativa y garantiza la viabilidad operativa y espacial del calendario universitario.

Palabras clave: gestión académica, detección de conflictos, arquitectura *full-stack*, *parsing*, coincidencia difusa.

Abstract

This project aims to design and develop a web application that facilitates the automatic detection of restrictions and conflicts in academic timetables, based on separate PDF documents containing the timetables and the course syllabi. The proposal seeks to automate a process that is currently often performed manually, which is error-prone and entails a high time cost for institutions.

To achieve this, the application allows users to upload official documents and extract structured information. Overcoming the limitations of conventional optical character recognition, the system implements a native parsing and geometric reconstruction engine, supported by fuzzy matching algorithms. Once the information is consolidated, the analytical core detects critical issues in real-time, such as class overlaps, double-booking of classrooms, or violations of teaching staff availability and scheduling constraints.

As a result, the project deploys an interactive, transactional, and scalable solution. By providing academic management with a systematic validation environment, the system not only eliminates logical inconsistencies in planning but also optimizes administrative efficiency and guarantees the operational and spatial viability of the university calendar.

Keywords: academic management, conflict detection, full-stack architecture, parsing, fuzzy matching.

Índice general

Listado de Acrónimos

1. Introducción	1
1.1. Objetivos	1
1.2. Ámbito y alcance	2
1.3. Estado del arte	3
2. Planificación y metodología	4
2.1. Metodología de desarrollo	4
2.2. Planificación temporal	5
2.3. Gestión de riesgos y herramientas	6
2.3.1. Entorno de desarrollo y tecnologías	6
2.3.2. Control de versiones y flujo de trabajo	7
2.3.3. Gestión de riesgos	8
2.4. Estudio de viabilidad económica y presupuesto	8
2.4.1. Costes de personal	8
2.4.2. Infraestructura, hardware y licencias	9
2.4.3. Resumen del presupuesto total	9
2.4.4. Conclusión de viabilidad	9
3. Ingeniería de requisitos y modelado del dominio	10
3.1. Identificación de actores	10
3.2. Especificación de requisitos de software	11
3.2.1. Requisitos funcionales	11
3.2.2. Requisitos no funcionales	13
3.3. Modelo conceptual	13
3.4. Casos de uso principales	14
4. Diseño del sistema	16
4.1. Arquitectura del software	16
4.2. Diseño de la base de datos	18
4.3. Diseño de la API REST	21
4.4. Diseño de la lógica de negocio	23
4.4.1. <i>Pipeline</i> de ingesta de fichas	24
4.4.2. <i>Pipeline</i> de ingesta de horarios	25
4.4.3. Motor de detección de conflictos	29
4.4.4. Dinámica de gestión de horarios	31
5. Implementación del sistema	35
5.1. Estructura y configuración	35
5.1.1. Organización física del repositorio	35
5.1.2. Gestión de la configuración y entornos	36

5.1.3.	Inicialización y <i>middleware</i>	36
5.2.	Capa de persistencia y acceso a datos	37
5.2.1.	Configuración del motor y conexiones	37
5.2.2.	Mapeo objeto-relacional	37
5.2.3.	Abstracción mediante repositorios	37
5.2.4.	Gestión transaccional y garantía ACID	38
5.3.	Implementación de la lógica de negocio y API	38
5.3.1.	API REST y validación de datos	38
5.3.2.	Implementación del flujo de ingesta de fichas	39
5.3.3.	Implementación del flujo de ingesta de horarios	39
5.3.4.	Motor de detección de conflictos	40
5.4.	Implementación de la interfaz de usuario	41
5.4.1.	Arquitectura de componentes y gestión de estado	41
5.4.2.	Implementación de la página principal y tableros	41
5.4.3.	Implementación de las páginas de ingesta	41
5.4.4.	Componente <i>grid</i> interactivo	42
5.4.5.	Integración visual de conflictos	42
6.	Validación y pruebas	43
6.1.	Validación de la lógica de negocio	43
6.1.1.	Verificación de los procesos de extracción y <i>parsing</i>	43
6.1.2.	Validación del motor de conflictos	44
6.1.3.	Pruebas de integridad de dominio y persistencia	45
6.2.	Validación del sistema e integración funcional	45
6.2.1.	Metodología de pruebas de caja negra	45
6.2.2.	Validación de la interacción y gestión de conflictos	46
6.2.3.	Estabilidad y usabilidad del sistema	46
6.3.	Pruebas de aceptación	46
7.	Conclusiones y trabajo futuro	47
A.	Ejemplo de ficha docente	50
B.	Ejemplo de cuadro horario	53

Listado de acrónimos

ACID	<i>Atomicity, Consistency, Isolation, Durability</i>
API	<i>Application Programming Interface</i>
CORS	<i>Cross-Origin Resource Sharing</i>
CRUD	<i>Create, Read, Update, Delete</i>
DTO	<i>Data Transfer Object</i>
ECTS	<i>European Credit Transfer and Accumulation System</i>
ERP	<i>Enterprise Resource Planning</i>
HTTP	<i>Hypertext Transfer Protocol</i>
JSON	<i>JavaScript Object Notation</i>
MIME	<i>Multipurpose Internet Mail Extensions</i>
OCR	<i>Optical Character Recognition</i>
ORM	<i>Object-Relational Mapping</i>
PDF	<i>Portable Document Format</i>
REST	<i>Representational State Transfer</i>
ROI	<i>Return on Investment</i>
SPA	<i>Single-Page Application</i>
SQL	<i>Structured Query Language</i>
TFG	Trabajo de Fin de Grado
TIC	Tecnologías de la Información y la Comunicación
UI	<i>User Interface</i>
UML	<i>Unified Modeling Language</i>
URL	<i>Uniform Resource Locator</i>
UX	<i>User Experience</i>

Capítulo 1

Introducción

La gestión académica en las instituciones de educación superior constituye un desafío logístico de gran envergadura. Específicamente, la confección de los horarios académicos representa un reto que se repite periódicamente en el calendario universitario. Esta labor de planificación resulta extremadamente laboriosa y compleja debido al alto volumen de variables que los gestores deben orquestar, satisfaciendo simultáneamente múltiples tipos de restricciones: desde la disponibilidad finita de recursos espaciales (aulas y laboratorios) y temporales, hasta los condicionantes derivados de la disponibilidad del profesorado, su conciliación y la coordinación con otras actividades académicas o intercentros.

En este escenario, la principal problemática detectada reside en la desconexión existente entre las dos fuentes de información que gobiernan la docencia:

1. **La planificación teórica:** documentada en las guías o fichas docentes, donde se define la estructura de la enseñanza y la distribución de horas lectivas (teoría, prácticas de laboratorio, seminarios).
2. **La ejecución temporal:** reflejada en los cuadros horarios y calendarios, que ubican esa planificación en el tiempo y el espacio.

Actualmente, en muchos centros, la conciliación entre ambas fuentes suele depender de procesos manuales o del uso de herramientas ofimáticas genéricas (hojas de cálculo), las cuales dificultan la detección automática de conflictos. Esta gestión manual introduce un margen de error humano significativo, permitiendo inconsistencias como la asignación de un docente en dos ubicaciones simultáneas o la sobreocupación de aulas. Una incoherencia en la planificación trasciende la mera inconsistencia de datos; sus consecuencias derivan en conflictos reales que, de no ser detectadas a tiempo, pueden generar interrupciones en la actividad lectiva y la consiguiente degradación de la calidad del servicio educativo.

La motivación impulsora de este trabajo de fin de grado (TFG) es ofrecer una solución tecnológica capaz de sustituir la validación humana —caracterizada por su ineficiencia operativa y su susceptibilidad al error— por una validación algorítmica y sistemática. El propósito final no es solo la digitalización de los datos, sino dotar a la gestión académica de una plataforma moderna que garantice la consistencia lógica de la planificación y facilite la toma de decisiones mediante una experiencia de usuario optimizada.

1.1. Objetivos

El objetivo general de este TFG consiste en el desarrollo de una aplicación web de arquitectura *full-stack* diseñada para la gestión, edición y auditoría automática de la planificación docente. El sistema se concibe como una plataforma integral de validación que permite la detección proactiva de inconsistencias, asegurando la coherencia entre la planificación teórica y la ejecución temporal de los horarios.

Para la consecución de esta meta, se definen los siguientes objetivos específicos:

- **Ingeniería de datos y normalización:** implementar herramientas de *parsing* capaces de procesar e interpretar información no estructurada o semi-estructurada proveniente de ficheros de fichas docentes y horarios preexistentes, transformándolos en un modelo de datos relacional coherente.
- **Desarrollo del motor de detección de conflictos:** construir la lógica de negocio central del sistema encargada de identificar automáticamente violaciones de integridad, tales como:
 - **Inconsistencias temporales (bilocación):** sesiones¹ simultáneas para un mismo docente.
 - **Inconsistencias espaciales:** solapamiento de grupos o asignaturas en un mismo recurso físico (aula).
 - **Incompatibilidades de disponibilidad docente:** violación de las restricciones horarias particulares definidas por el profesorado para su conciliación.
- **Implementación de herramientas de edición:** diseñar una interfaz de usuario que permita la gestión CRUD (*create, read, update, delete*) de la planificación, garantizando la consistencia de los datos en el momento de su introducción.
- **Optimización de la recuperación de información:** desarrollar algoritmos de coincidencia difusa (*fuzzy matching*) y heurísticas de aproximación para mitigar la ambigüedad en la ingesta de documentos fuente, asegurando la fiabilidad en la vinculación de entidades.
- **Garantía de persistencia y robustez:** diseñar una arquitectura de base de datos que asegure la integridad transaccional mediante el cumplimiento de las propiedades ACID², centralizando la información académica y eliminando la dispersión de datos.

1.2. Ámbito y alcance

El sistema desarrollado abarca el ciclo de vida completo de la gestión de la información académica, actuando como una plataforma de administración centralizada que integra desde la ingesta de fuentes documentales hasta la creación desde cero de horarios oficiales. El alcance funcional se restringe estrictamente a la lógica de dominio académico, priorizando la integridad de los datos y la detección algorítmica de conflictos.

Dada la naturaleza y el alcance de este proyecto, se han priorizado los módulos críticos de validación frente a funcionalidades administrativas secundarias. Los requisitos funcionales detallados y los casos de uso que componen el sistema se exponen en profundidad en el Capítulo 3. No obstante, con el fin de delimitar las fronteras técnicas del presente trabajo, se establecen las siguientes limitaciones y exclusiones:

- **Gestión de identidad y acceso:** el sistema se ha diseñado como una herramienta de administración de acceso único. No se contempla un sistema de autenticación multi-usuario o gestión de roles, asumiendo un entorno de confianza para el despliegue del prototipo.
- **Dependencia de la calidad del documento fuente:** la eficacia del módulo de ingesta de horarios está supeditada a la legibilidad y estructura de los archivos PDF (*portable document format*) de entrada. Ante desviaciones estructurales severas, la corrección final se delega al usuario mediante la interfaz de edición.

¹En el contexto del presente TFG, el término *sesión* define la unidad atómica de planificación temporal que representa una clase o actividad docente concreta, caracterizada por un intervalo de tiempo (día y horas), una ubicación física (aula) y un grupo de alumnos asociado.

²Acrónimo de *atomicity, consistency, isolation, durability*. Garantizan que las operaciones en la base de datos sean atómicas, consistentes, aisladas y duraderas, evitando la corrupción de datos ante fallos o concurrencia.

- **Resolución de conflictos:** el software actúa como un motor de auditoría y notificación; la resolución de las colisiones detectadas y la toma de decisiones final siguen recayendo en el criterio del gestor académico.

Finalmente, el código fuente del sistema desarrollado se encuentra disponible para su consulta técnica y validación en el repositorio oficial del proyecto³.

1.3. Estado del arte

En la actualidad, la gestión de la planificación académica en el entorno universitario se articula principalmente en torno a tres vertientes tecnológicas, cada una con limitaciones específicas que justifican el desarrollo de la solución propuesta en este proyecto:

- **Herramientas ofimáticas genéricas (hojas de cálculo):** es el método tradicional más extendido debido a su ubicuidad. Sin embargo, carece de una capa de lógica de negocio y de mecanismos de integridad transaccional. Al tratarse de una gestión puramente visual y manual, el riesgo de introducir errores de bilocación docente o solapamientos de aulas es crítico, ya que la herramienta no es capaz de interpretar la semántica de los datos ni de validar restricciones en tiempo real.
- **Software propietario y ERP⁴ educativos:** existen soluciones comerciales robustas para la gestión académica. No obstante, estas plataformas suelen presentar barreras significativas para los centros, tales como elevados costes de licencia, una rigidez estructural que dificulta su adaptación a normativas específicas de una facultad y curvas de aprendizaje pronunciadas que restan agilidad a la gestión diaria.
- **Sistemas de gestión *ad-hoc*:** algunas instituciones desarrollan soluciones internas, pero a menudo presentan una desconexión operativa entre la planificación teórica (fichas docentes) y la ejecución práctica (horarios).

Por tanto, existe un espacio técnico entre la flexibilidad pero inseguridad de las herramientas ofimáticas, y la robustez pero rigidez de los sistemas comerciales. La solución propuesta se ubica en este punto intermedio, priorizando la especialización de las reglas de negocio y la autonomía técnica de la facultad. Se busca, en definitiva, una transición de un modelo de revisión visual y reactiva hacia un modelo de gestión basado en la validación sistemática de los datos de planificación.

La presente memoria articula el proceso de ingeniería seguido a través de un recorrido estructural que comienza, tras esta introducción, con la exposición del marco metodológico e hitos temporales que han regido el proyecto, incluyendo un análisis pormenorizado de su viabilidad económica. Posteriormente, se profundiza en la fase de especificación y diseño, donde se definen los requisitos y la arquitectura del sistema, sirviendo de base para la descripción técnica de la implementación y el ecosistema tecnológico empleado. El documento concluye con la fase de validación, que verifica el cumplimiento de los objetivos planteados, y una síntesis final que recoge las conclusiones y las posibles líneas de evolución de la plataforma.

³Repositorio GitHub: <https://github.com/jma579/TFG>

⁴Siglas de *enterprise resource planning*. Es un tipo de software que las organizaciones utilizan para gestionar las actividades empresariales diarias.

Capítulo 2

Planificación y metodología

En este capítulo se detalla el marco organizativo y técnico que ha guiado el desarrollo del proyecto. Una correcta planificación en la ingeniería de software es fundamental no solo para el cumplimiento de los plazos temporales, sino para garantizar la calidad del producto final y la escalabilidad de su arquitectura.

A lo largo de las siguientes secciones, se describirá el enfoque metodológico adoptado —una estrategia híbrida fundamentada en el desarrollo por capas verticales— que ha permitido evolucionar el sistema de forma orgánica desde un núcleo funcional hasta una herramienta compleja de gestión académica. Asimismo, se expondrá la planificación temporal basada en evidencias reales de desarrollo, el desglose de las herramientas tecnológicas empleadas y, finalmente, un análisis del presupuesto y la viabilidad económica del proyecto. Este conjunto de apartados ofrece una visión integral de cómo se han transformado los requisitos teóricos en una solución técnica robusta y validada.

2.1. Metodología de desarrollo

El desarrollo de este proyecto se ha articulado bajo una metodología híbrida, iterativa e incremental, fundamentada en el prototipado evolutivo[1] y el desarrollo vertical (*vertical slicing*)[2]. A diferencia de los modelos lineales tradicionales, el ciclo de vida se caracterizó por una evolución bidireccional: mientras se establecía una base estructural sólida en las primeras etapas, las funcionalidades posteriores se implementaron de forma integral, atravesando todas las capas del sistema (base de datos, lógica de negocio y terminal de usuario).

Para comprender la naturaleza de este enfoque, es necesario precisar los pilares teóricos que lo sustentan:

- **Naturaleza híbrida, iterativa e incremental:** el software se construye mediante entregas funcionales que expanden las capacidades del sistema sobre una base validada. Esto permite gestionar la complejidad algorítmica de la aplicación de forma controlada.
- **Prototipado evolutivo:** esta técnica consiste en desarrollar una implementación inicial, exponerla al comentario del usuario (o a pruebas de estrés en este caso) y refinarla a través de múltiples versiones hasta que se convierte en el sistema final [3]. En este TFG, el prototipado permitió, por ejemplo, validar la lógica de extracción de datos antes de comprometer la arquitectura definitiva del *backend*.
- **Desarrollo vertical:** a diferencia del desarrollo por capas horizontales, el *vertical slicing* propone que cada iteración debe cruzar todas las capas de la arquitectura del software. Este enfoque garantiza que cada incremento aporte un valor funcional real y permite detectar problemas de integración de forma temprana, evitando que los errores de comunicación entre el *frontend* y el *backend* se acumulen hasta el final del proyecto.

Para garantizar la estabilidad durante este crecimiento incremental, se adoptó una estrategia de verificación y validación evolutiva, alineada con los protocolos detallados en el Capítulo 6 de esta memoria. Las evidencias en el sistema de pruebas del *backend* confirman un enfoque de madurez técnica organizado en tres niveles críticos:

- **Verificación técnica de bajo nivel:** pruebas de infraestructura iniciales para asegurar la integridad de la base de datos y la correcta carga de los modelos de datos.
- **Validación de la lógica de negocio:** verificación empírica de las reglas CRUD y de la precisión en la interpretación de los esquemas de datos extraídos.
- **Validación del motor de conflictos:** pruebas de aislamiento específicas para los algoritmos de detección de solapamientos, garantizando la fiabilidad de las alertas de severidad y el cumplimiento de las restricciones académicas.

2.2. Planificación temporal

La ejecución del proyecto se ha distribuido en un periodo de seis meses (agosto - enero), estructurándose en tres fases principales definidas por la madurez funcional del sistema y los hitos alcanzados en el repositorio de código.

Al inicio del proyecto, durante el mes de agosto y principios de septiembre, se realizó una investigación preliminar para definir la estructura lógica del sistema, lo que derivó en la creación de un *roadmap* técnico inicial enfocado exclusivamente en el *backend*. Este documento servía como guía de ruta para la construcción progresiva de los módulos de dominio y la infraestructura base.

Sin embargo, a medida que avanzaba el desarrollo, se hizo evidente que una planificación lineal y rígidamente predefinida no permitía responder con agilidad a los desafíos técnicos detectados, como la complejidad en el motor de extracción de horarios o la necesidad de validar la interfaz de usuario de forma temprana. Por ello, tras completar los módulos de dominio iniciales, se optó por evolucionar la planificación hacia una gestión basada en necesidades funcionales emergentes. Este cambio permitió priorizar hitos críticos no previstos originalmente —como la creación de prototipos *mock* y la reingeniería del extractor—, alineando la ejecución con la metodología de *vertical slicing* ya descrita.

1. **Fundamentos arquitectónicos y prototipado (agosto – octubre):** esta etapa inaugural se caracterizó por un desarrollo secuencial centrado en establecer los cimientos del sistema. El trabajo comenzó con el diseño del modelo de datos y la creación de la primera versión de la base de datos. Sobre esta estructura se construyó el servidor o *backend*, avanzando progresivamente desde la configuración de la infraestructura y la lógica central de negocio —incluyendo los extractores y el motor de conflictos— hasta la definición de los módulos de dominio. Una vez se dispuso de una interfaz de programación de aplicaciones (API) funcional preliminar, se procedió al desarrollo de la primera versión de la interfaz de usuario, la cual, apoyada en datos simulados (*mock data*), permitió validar la navegación y la coherencia visual antes de acometer la integración con los datos reales.
2. **Desarrollo vertical de flujos funcionales (noviembre – enero):** superada la etapa de prototipado, se adoptó una estrategia de desarrollo vertical. Bajo este enfoque, la implementación de cada nueva funcionalidad requirió el desarrollo conjunto y sincronizado de su lógica de negocio en el *backend* y su interfaz correspondiente en el *frontend*. Siguiendo este patrón integral, se abordó primero la gestión de recursos (aulas), para posteriormente iterar sobre los flujos de ingesta de fichas y horarios, incorporando en este último algoritmos avanzados de coincidencia difusa en el servidor y paneles de validación en el cliente. Finalmente, el ciclo se cerró con la construcción de las herramientas de interacción directa,

habilitando tanto la visualización y edición como la creación de nuevos horarios desde cero, asegurando en todo momento la coherencia entre la API y la experiencia de usuario.

3. **Integridad transaccional y lógica de conflictos (enero):** la etapa final del proyecto se orientó a la consolidación de la robustez y la inteligencia del sistema. En primer lugar, se acometió una refactorización profunda de la capa de persistencia para garantizar propiedades de atomicidad, consistencia, aislamiento y durabilidad, asegurando que las operaciones complejas de escritura generadas en la fase anterior mantuvieran la integridad de la base de datos.

Sobre estos cimientos reforzados se integró el motor de detección de conflictos. Dado que las funcionalidades de edición y creación implementadas en la fase 2 habían aumentado la casuística de uso, la incorporación de esta lógica de validación conllevó una necesaria reingeniería de los servicios existentes en el *backend*. Finalmente, esta lógica se trasladó a la interfaz de usuario, implementando las notificaciones visuales y los mecanismos de resolución que completan la experiencia funcional de la aplicación.

En el Cuadro 2.1, se presenta la representación visual de la dedicación temporal por tareas, reflejando el solapamiento de actividades propio del desarrollo iterativo.

Cuadro 2.1: Cronograma del proyecto estructurado por fases metodológicas

Fases y actividades	Ago	Sep	Oct	Nov	Dic	Ene
Fase 1: fundamentos arquitectónicos						
Diseño BD y configuración <i>backend</i>						
Lógica <i>core</i> y API REST						
Prototipo <i>frontend</i> (<i>mock data</i>)						
Fase 2: desarrollo vertical						
Módulo infraestructura (aulas)						
Flujo ingesta fichas (<i>back</i> + <i>front</i>)						
Flujo horarios (<i>Gridmaster</i> + UI)						
Herramientas edición y <i>fuzzy matching</i>						
Fase 3: consolidación y cierre						
Refactorización ACID y conflictos						

2.3. Gestión de riesgos y herramientas

La materialización de este sistema ha requerido una selección de herramientas coherente con las necesidades técnicas de cada etapa y una gestión de riesgos proactiva para garantizar la viabilidad del desarrollo. En esta sección se detallan los recursos tecnológicos empleados, justificando su elección en base a criterios de eficiencia, calidad de código y escalabilidad.

Más allá del *stack* tecnológico, se describe el enfoque adoptado para identificar y mitigar los desafíos técnicos que surgieron durante la implementación. Esta gestión de riesgos se centró en asegurar la integridad de la persistencia de datos ante operaciones complejas y en garantizar la precisión de los algoritmos de detección de conflictos, permitiendo que el sistema evolucionara de forma estable sin comprometer las funcionalidades ya validadas.

2.3.1. Entorno de desarrollo y tecnologías

La construcción del sistema se ha llevado a cabo en un entorno unificado basado en Visual Studio Code, configurado con extensiones de análisis estático para garantizar la homogeneidad

del código entre los distintos módulos. Para asegurar la reproducibilidad del software y evitar conflictos de versiones, se han aislado las dependencias de cada capa utilizando herramientas específicas de gestión de paquetes.

1. **Backend: ecosistema Python y FastAPI.** El servidor se ha desarrollado sobre Python 3.11, aprovechando las mejoras de rendimiento y las características avanzadas de tipado. La gestión del entorno se articula mediante las siguientes herramientas:

- **Gestión de dependencias:** se utiliza un entorno virtual nativo (`venv`) para aislar las librerías del sistema, manteniendo las versiones estables en el fichero `requirements.txt`.
- **Framework:** el núcleo del servidor es FastAPI¹ (v0.115), seleccionado por su soporte nativo para programación asíncrona (`async/await`) y su validación automática de datos basada en Pydantic.
- **Calidad y pruebas:** se ha integrado una suite de pruebas basada en `pytest` junto con `pytest-cov` para la monitorización de cobertura. Asimismo, se emplea `Ruff` para el formateo y `MyPy` para la verificación estática de tipos, asegurando la robustez de la lógica de negocio.

2. **Frontend: Next.js y React moderno.** La interfaz de usuario utiliza tecnologías de vanguardia sobre el ecosistema de React. A diferencia de la gestión tradicional con `npm` (*node package manager*), se ha optado por `pnpm` (*performant npm*) como gestor de paquetes. Esta elección permite optimizar la velocidad de instalación y la gestión de dependencias mediante el uso de enlaces simbólicos globales, reduciendo drásticamente el espacio en disco.

- **Arquitectura:** el proyecto se basa en Next.js 15² utilizando el *App Router*, permitiendo una gestión eficiente de *Server Components*. El motor de renderizado es React 19³, aprovechando las nuevas primitivas para la gestión de estados.
- **Lenguaje:** todo el código fuente está escrito en TypeScript en modo estricto, lo que garantiza la seguridad de tipos en tiempo de compilación.
- **Estilos y linting:** la maquetación se realiza mediante Tailwind CSS. Para la calidad del código, se utiliza ESLint con la nueva configuración plana (*flat config*) junto con Prettier, eliminando discrepancias estilísticas.

3. **Infraestructura de datos:** se ha optado por una base de datos relacional basada en SQLite para maximizar la agilidad durante el desarrollo. Esta decisión se apoya en el uso del ORM (*object-relational mapping*) SQLAlchemy⁴ (v2.0), que abstrae la capa de persistencia. Gracias a esta arquitectura, el sistema mantiene la capacidad de migrar a un motor de producción como PostgreSQL modificando únicamente la cadena de conexión, sin necesidad de refactorizar la lógica de la aplicación.

2.3.2. Control de versiones y flujo de trabajo

La gestión del código fuente se ha realizado mediante el sistema de control de versiones distribuido Git. El uso de esta herramienta ha sido clave no solo para mantener un historial de cambios detallado, sino también para facilitar la experimentación técnica sin comprometer la estabilidad de las versiones funcionales.

¹FastAPI: <https://fastapi.tiangolo.com/>

²Next.js: <https://nextjs.org/>

³React: <https://react.dev/>

⁴SQLAlchemy: <https://www.sqlalchemy.org/>

Para la organización del repositorio, se ha seguido una estrategia de ramificación que permitió aislar el desarrollo de nuevas funcionalidades. Especialmente relevante fue el uso de ramas independientes para la transición del *backend* original hacia la arquitectura actual; este enfoque permitió refactorizar módulos críticos y probar el nuevo motor de extracción de datos en paralelo, integrando los cambios en la rama principal solo tras verificar su estabilidad. Además, la trazabilidad de los *commits* ha servido como mecanismo de auditoría interna, permitiendo reconstruir la evolución cronológica del proyecto y recuperar estados previos ante errores de integración.

2.3.3. Gestión de riesgos

Una vez definido el entorno, se describen los desafíos técnicos identificados durante el desarrollo. Esta planificación no se concibió como un proceso estático, sino que se adaptó dinámicamente conforme las necesidades detectadas en las primeras etapas exigieron una reorientación de las estrategias iniciales. A continuación, se detallan los tres riesgos principales y sus medidas de mitigación:

1. **Complejidad en la lógica de detección de conflictos:** el riesgo de inestabilidad al manejar múltiples reglas de negocio (solapamientos horariales, disponibilidad de aulas y coincidencia de profesores) se mitigó mediante un desarrollo incremental. Se establecieron tres niveles de validación (*testing* evolutivo) que permitieron asegurar la precisión del motor de conflictos antes de su integración final con la interfaz de usuario.
2. **Inconsistencia en la extracción de datos desde PDF:** originalmente se contempló el uso de OCR (*optical character recognition*) para la lectura de documentos. Sin embargo, ante la falta de precisión detectada en las pruebas preliminares, se mitigó el riesgo mediante un cambio de estrategia: se descartó el reconocimiento óptico de caracteres en favor de un motor de parseo de texto nativo (*Gridmaster*). Si bien la extracción de datos desde documentos PDF presenta desafíos intrínsecos de estructura, esta solución demostró ser la que mejores resultados ofrecía para la casuística de los documentos oficiales utilizados en las pruebas, logrando una tasa de éxito significativamente superior a los métodos genéricos.
3. **Pérdida de integridad de datos por concurrencia:** las operaciones masivas de edición de horarios introducían el riesgo de dejar la base de datos en estados inconsistentes ante fallos inesperados. Para mitigar esto, se realizó una refactorización hacia una arquitectura ACID, asegurando que cualquier operación compleja de guardado se complete en su totalidad o se revierta automáticamente en caso de error, protegiendo la persistencia de la información.

2.4. Estudio de viabilidad económica y presupuesto

En esta sección se presenta el análisis de los costes asociados al desarrollo del proyecto. Este estudio es fundamental para determinar la viabilidad económica del sistema, tratando el TFG como un proyecto de ingeniería de software profesional.

2.4.1. Costes de personal

El cálculo de la mano de obra se basa en la dedicación real invertida desde agosto hasta enero, la cual asciende a un total estimado de 580 horas. Este volumen de trabajo refleja la alta complejidad del desarrollo *fullstack*, que incluye un motor de conflictos y más de 80 componentes de interfaz.

Se establece un precio por hora de 25,00 €⁵, lo que sitúa el valor del desarrollo en los estándares profesionales de la industria de software. En el Cuadro 2.2 se detalla la distribución de este esfuerzo según las diferentes fases de ingeniería del proyecto:

Cuadro 2.2: Desglose de costes de personal

Concepto / Fase	Horas	Precio/Hora	Subtotal
Análisis y diseño de arquitectura	80	25,00 €	2.000,00 €
Desarrollo <i>backend</i>	270	25,00 €	6.750,00 €
Desarrollo <i>frontend</i>	150	25,00 €	3.750,00 €
Aseguramiento de calidad y pruebas	80	25,00 €	2.000,00 €
Total costes de personal	580	-	14.500,00 €

2.4.2. Infraestructura, hardware y licencias

El desarrollo se ha realizado utilizando recursos propios y herramientas de código abierto, lo que reduce significativamente la inversión necesaria:

- **Hardware:** se estima la amortización de un equipo informático de gama media durante los 6 meses de uso, lo que supone un coste aproximado de 200,00 €.
- **Software:** el coste en licencias es de 0,00 €, debido al uso exclusivo de tecnologías *open source* (Python, FastAPI, Next.js, SQLite) y entornos de desarrollo gratuitos como Visual Studio Code.
- **Suministros:** se incluye una partida de 150,00 € en concepto de consumo eléctrico y conectividad a internet durante el periodo de ejecución.

2.4.3. Resumen del presupuesto total

Como resultado de la suma de los costes operativos y de personal, el presupuesto consolidado se presenta en el Cuadro 2.3, desglosando la base imponible y el impuesto sobre el valor añadido correspondiente:

Cuadro 2.3: Resumen total del presupuesto

Concepto	Importe
Costes de personal (580 horas)	14.500,00 €
Costes de infraestructura	350,00 €
Base imponible (subtotal)	14.850,00 €
IVA (21 %)	3.118,50 €
Total presupuesto	17.968,50 €

2.4.4. Conclusión de viabilidad

El coste total del proyecto asciende a 17.968,50 €. Teniendo en cuenta que el sistema automatiza procesos de gestión académica que actualmente requieren una alta carga de trabajo manual por parte de personal administrativo, la inversión se considera altamente viable. El retorno de la inversión (ROI) se produciría a corto plazo mediante el ahorro de costes indirectos, la optimización de los recursos docentes y la eliminación de errores humanos en la planificación de horarios.

⁵Cifra alineada con los convenios actuales del sector TIC y tarifas de mercado para perfiles *Junior*.

Capítulo 3

Ingeniería de requisitos y modelado del dominio

El propósito de este capítulo es definir con precisión el alcance funcional del sistema y las reglas que rigen su comportamiento. Tras haber establecido la planificación y viabilidad del proyecto en el capítulo anterior, esta fase se centra en la extracción, análisis y formalización de las necesidades detectadas en el ámbito de la gestión académica y la resolución de conflictos horarios.

El análisis que se presenta a continuación se realiza desde una perspectiva puramente conceptual. Siguiendo las buenas prácticas de la ingeniería de requisitos, se ha omitido cualquier decisión de implementación técnica para priorizar la comprensión lógica del problema. El objetivo principal es construir un modelo del dominio sólido que sirva de puente entre las necesidades del usuario final y el posterior diseño de la arquitectura del software.

A lo largo de las siguientes secciones se abordará:

- La identificación de los actores que interactúan con el sistema.
- La definición de los requisitos funcionales (capacidades del sistema) y no funcionales (restricciones de calidad).
- La representación de las entidades de información y sus relaciones.
- La descripción detallada de la interacción mediante casos de uso.

Esta base de conocimiento garantiza que el sistema resultante no solo sea técnicamente viable, sino que responda fielmente a la lógica de negocio requerida para la detección eficiente de conflictos en la planificación académica.

3.1. Identificación de actores

En esta sección se definen las entidades, humanas o externas, que interactúan con el sistema. Un actor representa un rol que un usuario o un sistema externo desempeña en relación con la aplicación de gestión académica.

Dada la naturaleza del sistema como una herramienta de soporte a la decisión y gestión técnica, se han identificado los siguientes actores:

- **Gestor académico (actor principal):** es el actor humano principal encargado de la administración y supervisión del modelo académico. Su interacción con el sistema es integral, siendo el responsable de iniciar los procesos críticos de negocio. Responsabilidades:
 - Gestión del catálogo: definir y mantener la estructura de los programas académicos (grados y másteres) y sus menciones.

- Supervisión de ingesta: cargar los documentos de referencia y validar que la extracción de datos sea correcta.
 - Gestión de recursos: administrar las aulas, grupos y la disponibilidad de los mismos.
 - Resolución de conflictos: ejecutar el motor de detección de colisiones horarias y marcar los conflictos como “resueltos” tras realizar los ajustes pertinentes en el sistema.
- **Sistema de información documental (actor secundario):** este actor representa el origen externo de los datos. No es una persona, sino un conjunto de fuentes de información técnica en formato PDF con las que el sistema interactúa para poblar su modelo de datos y reglas de negocio. Responsabilidades:
 - Provisión de estructura (fichas docentes): suministrar la información detallada de las asignaturas, carga de créditos y requisitos académicos.
 - Provisión de planificación (horarios): aportar los datos de sesiones, franjas horarias y asignaciones temporales.
 - Sincronización: actuar como la “fuente de la verdad” para el proceso de extracción, asegurando que la configuración del sistema coincide con la planificación oficial de la institución.
 - **Profesor (actor de dominio / pasivo):** aunque el profesorado no interactúa directamente con la interfaz, se define como un actor de dominio. Sus restricciones y preferencias son elementos fundamentales que el sistema procesa para la detección de conflictos. Responsabilidades:
 - Sujeto de restricciones: aportar (a través del gestor) las limitaciones de conciliación o disponibilidad que el sistema debe validar para evitar solapamientos.

3.2. Especificación de requisitos de software

La especificación de requisitos establece el marco funcional y de calidad del sistema. Se presenta a continuación una síntesis detallada de las capacidades del software, validada contra la implementación final.

3.2.1. Requisitos funcionales

Módulo de gestión de fichas (guías docentes): este módulo centraliza la ingesta y normalización de la información académica teórica.

- **RF-FIC-01 (Procesamiento de archivos):** el sistema debe permitir la carga individual o masiva (*batch*) de archivos PDF, validando su legibilidad y la presencia de texto extraíble antes de iniciar el *pipeline* de extracción.
- **RF-FIC-02 (Extracción y estructuración):** el sistema debe realizar el *parsing* de los documentos para almacenar de forma estructurada datos de asignaturas (créditos, temporalidad, modalidad), infiriendo incluso información implícita como la titulación.
- **RF-FIC-03 (Sincronización e idempotencia):** el sistema debe tratar el PDF como fuente de verdad (*source of truth*). En cargas sucesivas, actualizará registros existentes en lugar de duplicarlos y gestionará de forma diferencial la vinculación de profesores.
- **RF-FIC-04 (Persistencia e integridad):** el sistema debe garantizar la creación automática de entidades relacionadas (programas, profesores) y asegurar la integridad transaccional mediante mecanismos de *rollback* ante errores.

- **RF-FIC-05 (Gestión administrativa):** el sistema debe proporcionar informes visuales de los resultados de la ingesta y permitir la eliminación manual de registros para correcciones excepcionales.

Módulo de gestión de horarios: regula la digitalización de la planificación temporal mediante aprendizaje asistido.

- **RF-HOR-01 (Digitalización de cuadros horarios):** el sistema debe analizar la estructura visual de horarios en PDF para identificar tramos, días, grupos y aulas.
- **RF-HOR-02 (Normalización mediante aprendizaje):** el sistema debe proponer vínculos mediante *fuzzy matching* y registrar automáticamente como *alias* aquellas sugerencias que sean validadas por el usuario (al no ser modificadas), optimizando así la precisión de futuras importaciones.
- **RF-HOR-03 (Validación en *staging area*):** antes de la persistencia, el sistema debe ofrecer una interfaz de previsualización editable para auditar y confirmar los datos extraídos.
- **RF-HOR-04 (Ciclo de vida de sesiones):** el sistema debe permitir la creación manual, edición granular (ubicación, docente, franja) y eliminación en cascada de horarios y sus sesiones asociadas.
- **RF-HOR-05 (Salida de datos):** el sistema debe optimizar la visualización de los horarios para su impresión directa o exportación documental desde el navegador.

Módulo de detección de conflictos: núcleo de validación que garantiza la viabilidad de la planificación académica.

- **RF-CON-01 (Tipología de colisiones):** el sistema debe detectar automáticamente conflictos espaciales (aulas), de ubicuidad docente (profesores), de conciliación horaria y académicos (solapamientos en el mismo grupo).
- **RF-CON-02 (Lógica de evaluación):** el sistema debe calcular solapamientos reales en cualquier intervalo temporal, clasificando las incidencias por niveles de severidad y asignándoles un identificador único (*hash*).
- **RF-CON-03 (Ejecución preventiva y persistente):** la detección debe ejecutarse automáticamente post-ingesta, de forma preventiva durante la edición (simulación) y de manera obligatoria al confirmar cualquier cambio.

Módulo transversal de interfaz de usuario (*user interface*, UI): optimiza la interacción del gestor con el volumen de datos académicos.

- **RF-UI-01 (Navegación y control):** el sistema debe presentar un tablero de mando (*dashboard*) con métricas clave y un menú lateral persistente para conmutar entre módulos sin pérdida de contexto.
- **RF-UI-02 (Gestión avanzada de tablas):** las vistas de datos deben integrar búsqueda en tiempo real, filtrado multivariable, ordenación bidireccional y filas expandibles para detalles jerárquicos.
- **RF-UI-03 (Feedback y estados):** el sistema debe comunicar estados mediante notificaciones (*toasts*), indicadores de carga asíncrona (*spinners*), distintivos de color (*badges*) y componentes informativos para búsquedas vacías.

- **RF-UI-04 (Componentes de interacción):** el uso de ventanas modales, validación de formularios en tiempo real y retroalimentación sobre el progreso de subida de archivos debe priorizar la fluidez de la experiencia.

3.2.2. Requisitos no funcionales

- **RNF-EF-01 (Rendimiento y asincronía):** la interfaz debe responder en menos de 2 segundos. Las operaciones costosas (PDF de hasta 20 MB) deben ejecutarse de forma no bloqueante mediante patrones asíncronos.
- **RNF-US-01 (Diseño y accesibilidad):** el sistema debe ser adaptativo (*responsive*), mantener una consistencia visual unificada para reducir la carga cognitiva y seguir buenas prácticas de accesibilidad técnica.
- **RNF-MA-01 (Calidad y estándares):** el código debe adherirse a estándares de legibilidad (PEP 8¹, *type hints*² y *ESLint*³), utilizar tipado estricto y seguir una arquitectura modular desacoplada.
- **RNF-RE-01 (Restricciones técnicas):** el ecosistema se restringe al uso de Python y React para garantizar la potencia de procesamiento y la seguridad en el tipado, empleando un motor de persistencia ligero para facilitar la portabilidad.

3.3. Modelo conceptual

El modelo conceptual constituye la representación abstracta de la realidad académica sobre la que opera el sistema. Su objetivo es identificar las entidades de información fundamentales, sus atributos de negocio y las relaciones que gobiernan la lógica de la institución, independientemente de cualquier consideración técnica de persistencia.

Este modelo se estructura en cuatro dominios lógicos que permiten desacoplar las responsabilidades del sistema y facilitar la comprensión de la gestión académica:

- **Catálogo académico:** define la estructura estática y curricular, estableciendo cómo se organizan las enseñanzas y los planes de estudio.
- **Gestión de recursos:** administra los activos disponibles, tanto humanos (personal docente) como físicos (aulas), necesarios para el desarrollo de la actividad educativa.
- **Planificación docente:** representa el núcleo dinámico del sistema donde se materializan los horarios mediante la asignación de recursos en franjas temporales concretas.
- **Motor de reglas:** supervisa la integridad del modelo mediante la detección y persistencia de conflictos derivados de colisiones físicas o lógicas en la planificación.

La formalización de este dominio garantiza que el sistema sea capaz de gestionar escenarios complejos, como la co-docencia o la compartición de asignaturas entre distintos programas, manteniendo siempre una fuente única de verdad basada en la documentación oficial.

Para representar la arquitectura de la información, se ha elaborado un diagrama de clases de análisis (véase la Figura 3.1) que ilustra la jerarquía y las dependencias entre las entidades del sistema. Este modelo no debe interpretarse como un esquema físico de base de datos, sino como el conjunto de conceptos de negocio que permiten la gestión académica y la resolución de conflictos.

¹PEP 8: <https://peps.python.org/pep-0008/>

²Type hints: <https://docs.python.org/3/library/typing.html>

³ESLint: <https://eslint.org/>

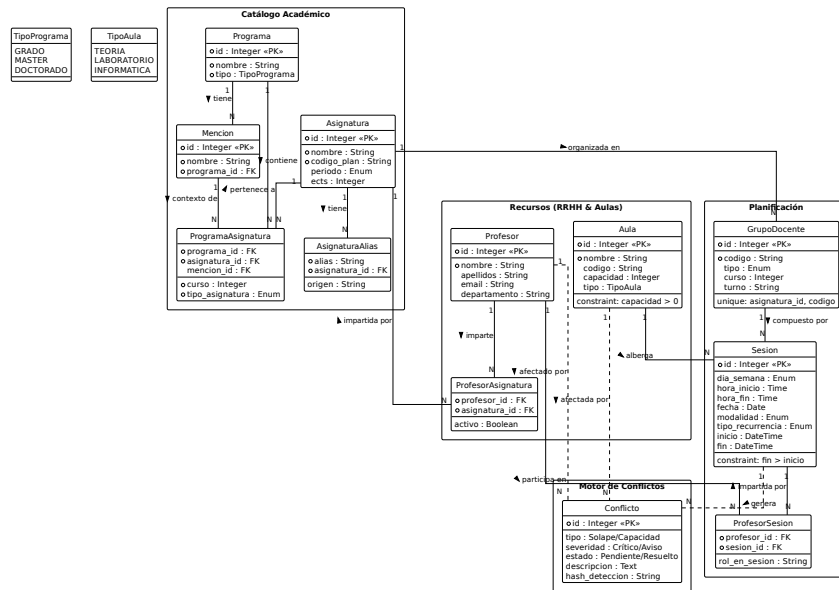


Figura 3.1: Diagrama de clases de análisis organizado por dominios lógicos.

- **Descripción de entidades y conceptos de negocio:** en el ámbito del catálogo, el **Programa** representa la titulación oficial y la **Mención** sus ramas de especialización. La **Asignatura** actúa como referencia académica (definida por su código de plan y créditos), mientras que el **AsignaturaAlias** normaliza nombres variantes en la denominación de fuentes externas funcionando como un mecanismo de aprendizaje continuo y **ProgramaAsignatura** establece su ubicación curricular (curso y tipo). Respecto a los recursos, el **Profesor** y el **Aula** definen el capital humano y físico disponible. El núcleo operativo reside en el **GrupoDocente**, que organiza a los alumnos por turno y modalidad, y en la **Sesión**, evento donde confluyen grupo, aula y docente en un tiempo determinado. Finalmente, el **Conflicto** persiste las colisiones detectadas por el motor de reglas para garantizar su trazabilidad.
- **Reglas y restricciones del dominio:** el sistema garantiza la integridad del modelo mediante el cumplimiento de diversas invariantes de negocio. Se impone una coherencia cronológica que impide que un evento finalice antes de su inicio y una consistencia física que exige aforos positivos en las aulas. La unicidad de la planificación prohíbe grupos duplicados por asignatura, mientras que la idempotencia del motor de reglas evita reportes redundantes mediante el uso de *hashes* únicos. Por último, se asegura la integridad referencial mediante borrados en cascada para evitar registros huérfanos y se exige una validación de recursos mínima, obligando a que toda sesión esté vinculada a un aula y un grupo docente para ser válida.

3.4. Casos de uso principales

El análisis funcional del sistema se ha modelado utilizando el estándar UML⁴ para formalizar las interacciones entre los usuarios y la aplicación, delimitando el alcance del sistema y describiendo cómo los diferentes módulos responden a los eventos disparados por el actor principal.

En la Figura 3.2 se presenta la visión global de la solución, centrada en la figura del coordinador académico. Este actor orquesta el flujo de datos desde la ingesta de guías docentes hasta la resolución de conflictos, apoyándose en procesos automáticos de validación representados mediante relaciones de inclusión («*include*») en los flujos de importación.

⁴UML (*unified modeling language*): Estándar para el modelado de sistemas de software <https://www.uml.org/>

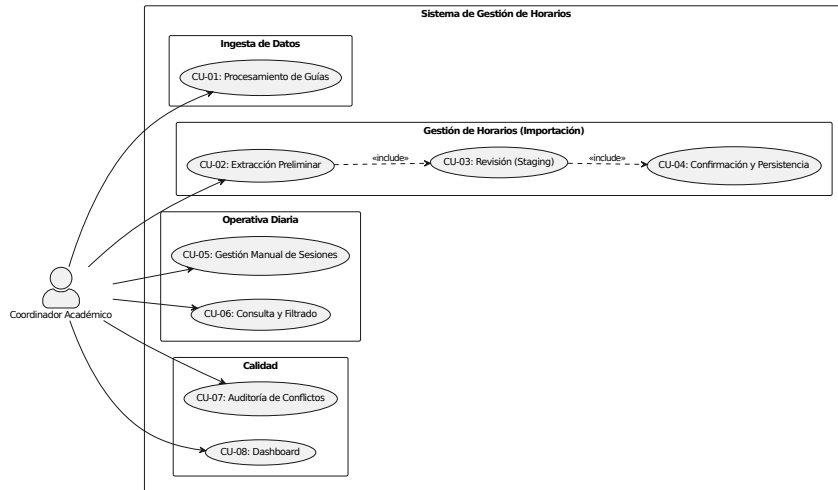


Figura 3.2: Diagrama general de casos de uso del sistema organizado por módulos operativos.

A continuación, se desglosan los casos de uso principales clasificados por su dominio funcional:

Módulo de catálogo académico (ingesta): gestiona la alimentación de datos maestros transformando documentos no estructurados en entidades relacionales.

- **CU-01 (Procesamiento de guías docentes):** permite al actor extraer automáticamente metadatos (códigos, créditos, docentes) mediante técnicas de análisis semántico, garantizando la actualización de la base de datos de forma idempotente.

Módulo de gestión de horarios (importación): este bloque abarca el ciclo de vida de la digitalización de horarios mediante un flujo secuencial crítico.

- **CU-02 (Extracción preliminar):** análisis visual del PDF para detectar la estructura de la rejilla horaria y proponer un borrador inicial mediante algoritmos de coincidencia difusa.
- **CU-03 (Revisión y corrección en *staging*):** interfaz intermedia donde el usuario valida la calidad de la extracción y corrige manualmente las asociaciones de baja confianza.
- **CU-04 (Confirmación y persistencia):** volcado definitivo de los datos validados a la base de datos operativa y ejecución inmediata del motor de conflictos.

Módulo de operativa diaria: funcionalidades destinadas al mantenimiento continuo y la consulta de la planificación.

- **CU-05 (Gestión manual de sesiones):** permite la creación, edición y borrado de sesiones individuales directamente sobre la interfaz del calendario para realizar ajustes finos.
- **CU-06 (Consulta y filtrado):** visualización dinámica del horario académico filtrado por criterios como titulación, curso o grupo docente.

Módulo de calidad y supervisión: herramientas transversales para la monitorización del estado de salud de los horarios.

- **CU-07 (Auditoría de conflictos):** gestión y visualización detallada de las incidencias detectadas, permitiendo al actor marcar estados de resolución o ignorar alertas.
- **CU-08 (*Dashboard*):** panel centralizado que permite la visualización de los horarios existentes y proporciona una vista rápida de los conflictos asociados al horario.

Capítulo 4

Diseño del sistema

El presente capítulo constituye el núcleo técnico de la propuesta, donde se traslada el modelo conceptual y los requisitos funcionales definidos previamente hacia una arquitectura de software robusta, escalable y mantenible. El objetivo principal del diseño es establecer el “cómo” del sistema, definiendo la estructura lógica, los mecanismos de persistencia y los protocolos de comunicación que permitirán dar respuesta a la problemática de la gestión de horarios y detección de conflictos académicos.

Es importante diferenciar el alcance de este capítulo respecto a la fase de implementación posterior. Mientras que el diseño aquí expuesto se centra en la definición de estructuras, contratos de interfaz y flujos lógicos abstractos, el capítulo siguiente abordará la materialización de dichos diseños mediante lenguajes de programación y entornos de ejecución. Por tanto, este apartado actúa como el “plano maestro” que garantiza la coherencia arquitectónica independientemente de las particularidades del código.

Para ello, el capítulo profundiza inicialmente en la definición de una arquitectura desacoplada que fundamenta la separación entre la presentación y la lógica, estableciendo el marco estructural del sistema. A continuación, se detalla el diseño físico de la base de datos y los contratos de interfaz que rigen la API REST, elementos esenciales para garantizar la persistencia y el intercambio de información. Finalmente, se modela la lógica de negocio, haciendo especial énfasis en los procesos de ingesta y en el funcionamiento algorítmico del motor de detección de conflictos, concluyendo con el diseño de la interfaz de usuario orientado a la gestión eficiente de la información académica.

4.1. Arquitectura del software

El diseño del sistema se basa en una arquitectura de estilo cliente-servidor desacoplada, articulada mediante el protocolo de transferencia de estado representacional (REST). Esta decisión estructural responde a la necesidad de separar las responsabilidades de presentación de la lógica de negocio pura, permitiendo que ambos dominios evolucionen de forma independiente y facilitando la escalabilidad horizontal del sistema. La arquitectura garantiza que el cliente sea totalmente agnóstico a la implementación de la persistencia, interactuando con el servidor exclusivamente a través de contratos de datos en formato JSON. Este enfoque de diseño modular permite una gestión eficiente de la complejidad, asegurando que el motor de detección de conflictos y los servicios de ingesta de datos operen en un entorno aislado y controlado, optimizando así el rendimiento global de la aplicación.

Como representación de esta organización lógica y física, se presenta a continuación el diagrama de contenedores basado en el modelo C4 (véase la Figura 4.1), el cual ilustra la jerarquía de los componentes y los flujos de comunicación entre los distintos actores y servicios del sistema.

Diagrama de Contenedores - Sistema de Planificación Docente

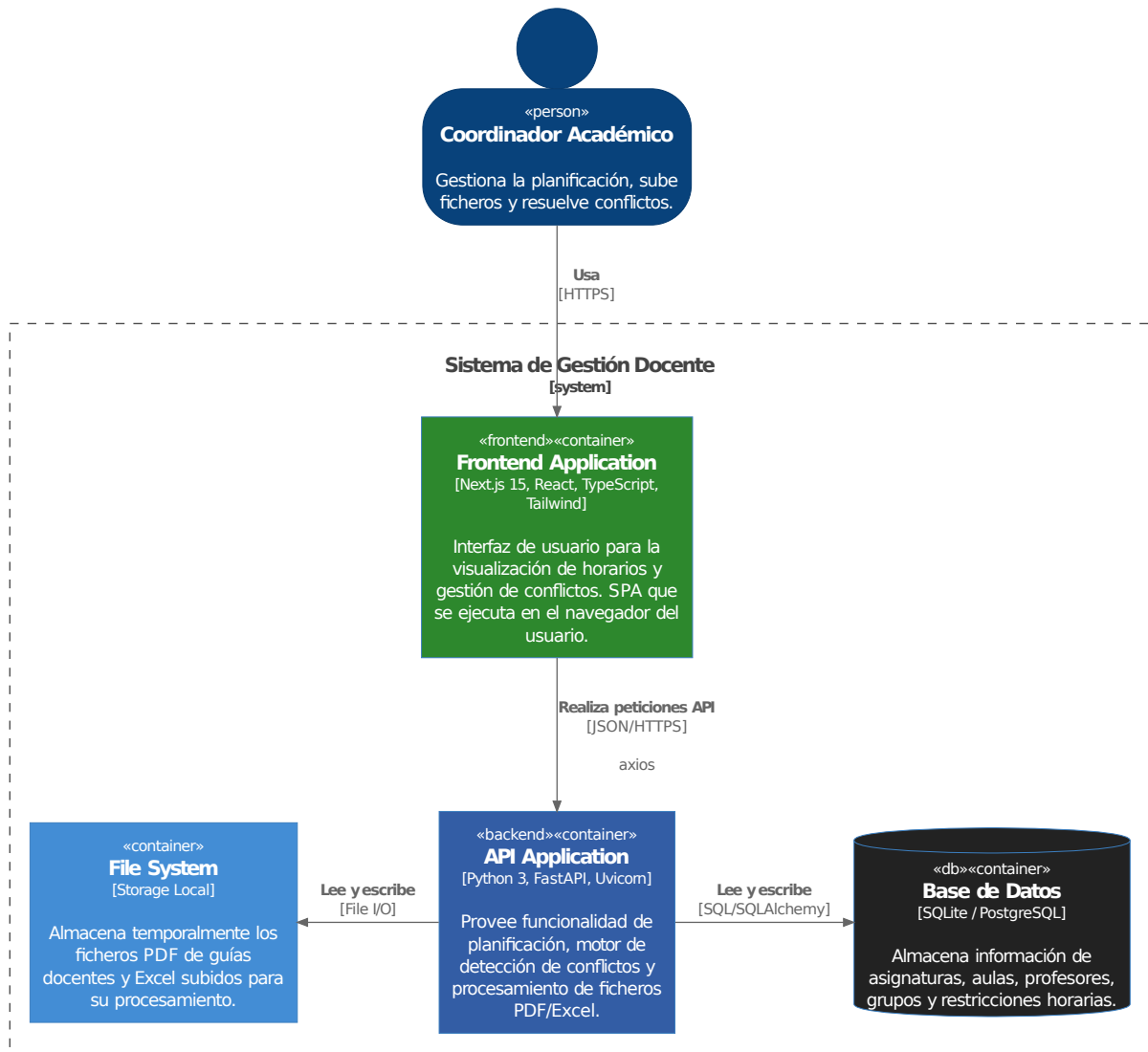


Figura 4.1: Diagrama de contenedores mostrando la arquitectura técnica y el flujo de comunicación.

A partir de la arquitectura definida, se describen de forma detallada los componentes fundamentales que integran el sistema:

- **Frontend application:** se trata de la interfaz de usuario encargada de la capa de presentación. El diseño utiliza una arquitectura SPA (*single-page application*) con una estrategia de renderizado híbrido, lo que permite delegar parte del procesamiento de la interfaz al servidor para optimizar la velocidad de respuesta inicial. Su responsabilidad principal reside en la visualización dinámica de horarios, la gestión del estado de la interacción y la orquestación de las peticiones hacia la API para la obtención y envío de datos. Para asegurar una experiencia de usuario fluida, esta capa implementa un sistema de gestión de estado local que permite la reactividad inmediata ante la detección de conflictos, facilitando al gestor académico la toma de decisiones sin tiempos de espera prolongados.
- **API application:** este componente constituye el núcleo del *backend*, implementando una arquitectura orientada al dominio, organizando el código en módulos funcionales en lugar de capas técnicas, lo que facilita su mantenibilidad. En su interior residen los motores

más complejos del sistema: el proceso de ingesta, encargado de la transformación de archivos no estructurados (PDF) en entidades lógicas, y el motor de detección de conflictos, que ejecuta algoritmos de validación espacio-temporal sobre la disponibilidad de aulas y profesorado. Este servidor actúa como mediador central, validando cada transacción antes de su persistencia para garantizar que las reglas de negocio académicas se cumplan rigurosamente en cada operación.

- **Base de datos:** representa la capa de persistencia para los datos estructurados del sistema, tales como asignaturas, grupos y restricciones. Se utiliza un motor relacional (SQL) gestionado a través de una capa de abstracción de mapeo objeto-relacional. Esta abstracción permite que la lógica de negocio sea independiente del motor de base de datos específico utilizado, asegurando una transición transparente entre entornos de desarrollo y producción. El diseño del esquema se ha optimizado mediante la creación de índices estratégicos en columnas de alta consulta, como códigos de asignatura y periodos lectivos, mejorando así la eficiencia de los algoritmos de búsqueda de colisiones.
- **File system:** actúa como un almacenamiento de persistencia física para la gestión de documentos. Este componente es esencial para el flujo de trabajo de ingesta, funcionando como un repositorio temporal donde se depositan los archivos de guías docentes y horarios antes de ser procesados, extraídos y normalizados por la lógica del *backend*. Se ha diseñado un esquema de nomenclatura única y una estructura de directorios jerárquica para evitar colisiones de archivos y facilitar la trazabilidad entre el registro en base de datos y el documento físico original.
- **Comunicación e intercambio de datos:** el flujo de información entre el cliente y el servidor se realiza estrictamente bajo el protocolo HTTPS. La seguridad en el intercambio de recursos se garantiza mediante políticas de origen cruzado (CORS), que restringen las operaciones únicamente a dominios autorizados, mientras que la integridad de los datos se mantiene mediante la validación de esquemas en la serialización y deserialización de los objetos de transferencia de datos. Este intercambio asíncrono asegura que la interfaz de usuario permanezca operativa incluso durante procesos de cálculo intensivos en el servidor.

4.2. Diseño de la base de datos

El diseño de la persistencia se fundamenta en un modelo físico relacional, cuya estructura ha sido proyectada para garantizar la integridad, la consistencia y la disponibilidad de la información académica[4]. Un modelo relacional organiza los datos en tablas vinculadas mediante claves, asegurando que las dependencias entre entidades cumplan con estrictas reglas de integridad referencial. Se ha optado por esta arquitectura en detrimento de soluciones no relacionales debido a la naturaleza altamente interconectada y estructurada de los datos gestionados: la planificación de horarios y la detección de conflictos requieren de una consistencia inmediata y de la capacidad de ejecutar consultas complejas con múltiples uniones entre tablas. Mientras que las bases de datos de documentos priorizan la flexibilidad del esquema y la escalabilidad horizontal, el presente proyecto demanda la robustez que ofrecen las transacciones ACID para evitar anomalías como la asignación duplicada de un aula o el solapamiento de horarios de un profesor, situaciones que en modelos no relacionales podrían derivar en estados de datos inconsistentes.

Para alcanzar estos objetivos, el esquema se ha normalizado siguiendo la tercera forma normal, eliminando redundancias y garantizando que cada pieza de información se almacene en un único lugar lógico. Esta normalización es crítica para el motor de conflictos, ya que asegura que cualquier actualización en la disponibilidad de un recurso se refleje instantáneamente en todas las consultas dependientes. Asimismo, se han implementado restricciones de integridad a nivel de esquema, tales como verificaciones de rangos horarios y unicidad de códigos académicos, lo que constituye una primera barrera de defensa contra la entrada de datos corruptos o incoherentes.

Para la representación visual de esta arquitectura de datos, se ha elaborado un diagrama entidad-relación que sirve como plano maestro para la organización de la base de datos. En la Figura 4.2 se ilustran las entidades principales, sus atributos y la cardinalidad de sus vínculos, proporcionando una visión integral de cómo se estructuran y relacionan los distintos módulos funcionales.

Figura 4.2: Diagrama entidad-relación del sistema de gestión horaria.

- **Catálogo académico:** representa la estructura base de la oferta educativa, definiendo la jerarquía y las características de las materias que se imparten.
 - **Programa:** entidad que define los planes de estudio del centro. Almacena el nombre oficial y el tipo de titulación (grado o máster), garantizando mediante restricciones de unicidad que no existan duplicidades para un mismo tipo de formación.
 - **Mención:** representa las especializaciones o itinerarios disponibles dentro de un programa. Actúa como un nivel intermedio de clasificación que permite agrupar asignaturas específicas de una orientación académica concreta.
 - **Asignatura:** es la unidad docente fundamental. Además de su nomenclatura y código identificativo, registra metadatos críticos como el periodo lectivo (semestre), la modalidad (presencial o híbrida), el idioma de impartición y el volumen de créditos. Incluye validaciones para asegurar que el curso académico sea un valor positivo coherente.
 - **ProgramaAsignatura:** tabla de asociación que resuelve la relación de muchos a muchos entre planes de estudio y materias, permitiendo que una misma asignatura pueda ser compartida por diferentes programas o menciones.

- **Recursos:** gestiona los elementos humanos y físicos necesarios para el desarrollo de la actividad docente, así como las condiciones que rigen su disponibilidad.
 - **Profesor:** almacena la información del personal docente, incluyendo sus datos de contacto y departamento. En esta entidad se integra la lógica de conciliación, mediante indicadores que señalan si el profesional requiere de ajustes horarios específicos, centralizando así las restricciones de disponibilidad directamente en el perfil del docente.
 - **Aula:** representa los espacios físicos donde se imparte la docencia. Define la capacidad máxima de alumnos y el tipo de instalación (teoría o laboratorio). El diseño incluye una restricción de verificación para garantizar que el aforo sea siempre un número positivo.
 - **ProfesorAsignatura:** tabla técnica que vincula a los docentes con las materias que tienen previsto impartir.
- **Planificación docente:** constituye el núcleo operativo del sistema, donde se materializa la asignación de tiempos y espacios.
 - **GrupoDocente:** define las agrupaciones de alumnos para una asignatura. Clasifica los grupos según su tipología (teoría o práctica) y el idioma específico de ese conjunto de estudiantes, vinculándose directamente con la asignatura matriz.
 - **Sesión:** representa cada una de las entradas en el calendario académico. Vincula de forma unívoca un grupo con un aula y un profesor en un intervalo temporal definido (hora de inicio y fin) y un día de la semana específico. Incluye lógica de integridad para asegurar que la hora de finalización sea siempre posterior a la de inicio.
 - **Profesor-Sesión:** permite la gestión de docencia compartida, habilitando que múltiples profesores puedan estar asignados a una única sesión horaria si la planificación así lo requiere.
- **Motor de conflictos:** este módulo se encarga de la persistencia de las inconsistencias detectadas por los algoritmos de validación del sistema.
 - **Conflicto:** tabla destinada al registro de colisiones lógicas. Almacena el tipo de conflicto (solapamiento de aula, doble asignación de profesor o incumplimiento de conciliación), la severidad del mismo y su estado actual (pendiente o resuelto). Al referenciar las sesiones implicadas, permite que la interfaz de usuario recupere y presente las alertas de forma contextualizada al gestor académico.

Cabe precisar que el diseño incluye las entidades de trazabilidad de ingesta y la tabla de restricciones genéricas como una previsión para su implementación en futuras iteraciones del sistema. Actualmente, la trazabilidad se gestiona de forma directa sobre el sistema de ficheros para simplificar el flujo operativo, mientras que la conciliación docente se ha integrado como un atributo directo en la entidad profesor para optimizar las consultas del motor de conflictos en la etapa actual de desarrollo.

Para garantizar la coherencia del sistema, el diseño físico incorpora un conjunto de mecanismos de integridad y reglas de dominio que actúan como la primera capa de validación de la lógica de negocio. Estas restricciones aseguran que los datos persistidos cumplan siempre con los requisitos funcionales definidos para la gestión académica:

- **Integridad referencial:** se han definido claves foráneas con políticas de borrado y actualización controladas. Por ejemplo, la eliminación de una asignatura implica un borrado en cascada de sus grupos y sesiones asociados, mientras que la eliminación de un aula está restringida si existen sesiones vinculadas, evitando así la existencia de horarios huérfanos de recursos físicos.

- **Restricciones de unicidad:** el modelo impone que ciertos atributos o combinaciones de estos sean irrepetibles. Esto impide la creación de dos programas con el mismo nombre y tipo, la duplicidad de correos electrónicos en el profesorado o la existencia de dos grupos con el mismo código para una misma asignatura.
- **Verificaciones de dominio (*check constraints*):** se han implementado reglas de validación aritmética y lógica directamente en el esquema. Entre ellas destacan la obligatoriedad de que la capacidad de las aulas sea un valor positivo y la restricción temporal que exige que la hora de fin de cualquier sesión o periodo de conciliación sea estrictamente posterior a su hora de inicio.
- **Coherencia de estados:** mediante el uso de tipos enumerados, el diseño limita los valores posibles para campos críticos como la severidad de los conflictos, el periodo lectivo o el idioma de impartición, garantizando que el motor de detección procese únicamente categorías de datos predefinidas y válidas.

4.3. Diseño de la API REST

El intercambio de información entre el cliente y el servidor se articula mediante una interfaz de programación de aplicaciones basada en el estilo arquitectónico REST[5]. Este modelo de diseño se define como un conjunto de principios que permiten la creación de servicios web ligeros, altamente escalables y, fundamentalmente, sin estado (*stateless*), lo que significa que cada petición del cliente contiene toda la información necesaria para ser procesada por el servidor de forma independiente. Se ha seleccionado este enfoque debido a su capacidad para desacoplar completamente la capa de presentación de la lógica de negocio; esta independencia permite que el *frontend* y el *backend* evolucionen de manera autónoma, facilitando la integración con diferentes tipos de clientes y optimizando el consumo de recursos al evitar el mantenimiento de sesiones complejas en el servidor.

La interfaz actúa como un contrato de comunicación formal que define cómo se exponen los recursos del sistema y qué protocolos deben seguirse para su manipulación. El diseño se fundamenta en el uso del protocolo HTTPS para garantizar la seguridad y privacidad de los datos académicos en tránsito, empleando JSON como formato universal de serialización debido a su ligereza y su capacidad para representar de forma estructurada las relaciones complejas de la planificación docente. La API se organiza en torno a recursos únicos identificados mediante URLs, donde la interacción se rige por la semántica de los verbos HTTP estándar: GET para la recuperación de información, POST para la creación de entidades, PUT o PATCH para la actualización de registros y DELETE para su eliminación.

Modelos de transferencia de datos

La comunicación entre el cliente y el servidor se sustenta en el uso de objetos de transferencia de datos (*data transfer objects*, DTOs), los cuales actúan como esquemas lógicos que definen la estructura exacta de la información en cada transacción. Este diseño permite desacoplar los modelos de persistencia de la base de datos de los modelos de visualización del *frontend*, garantizando que solo se transfiera la información estrictamente necesaria y protegiendo la integridad del sistema ante la recepción de datos mal formados o incompletos. Cada recurso de la API cuenta con tres variantes principales de contratos de datos para cubrir el ciclo de vida de la información:

- Modelos de creación: definen los atributos mínimos obligatorios que el gestor académico debe proporcionar para dar de alta un nuevo recurso, como los datos identificativos de una asignatura o los parámetros iniciales de una sesión.

- Modelos de actualización: permiten la modificación de los registros existentes, diseñados para soportar tanto cambios integrales como actualizaciones parciales de campos específicos, como la capacidad de un aula o el estado de un conflicto.
- Modelos de respuesta: representan la estructura de los datos que el servidor devuelve tras procesar una petición. Estos contratos incluyen metadatos generados automáticamente por el sistema, tales como identificadores únicos, marcas temporales de creación y relaciones con otros recursos del sistema que no están presentes en los modelos de entrada.

Un componente fundamental de este diseño es la validación automática de esquemas. Antes de que una petición alcance la lógica de negocio, la capa de la API verifica que el cuerpo del mensaje cumpla con las restricciones de tipo, longitud y obligatoriedad definidas en el contrato. En caso de detectarse una discrepancia, el sistema interrumpe el flujo y devuelve una respuesta estructurada que detalla el error cometido, permitiendo que la interfaz de usuario informe de manera precisa al usuario sobre la corrección necesaria.

Catálogo de recursos

Los puntos de acceso de la API se organizan en módulos lógicos que agrupan las operaciones relacionadas con una misma área funcional del sistema. Esta organización modular permite que la interfaz de usuario interactúe con el *backend* de forma granular, solicitando únicamente los recursos necesarios para cada tarea de gestión.

Cada recurso expone una serie de operaciones estándar que permiten al gestor académico realizar acciones de consulta, creación o modificación sobre los datos del dominio. En el Cuadro 4.1 se detallan los recursos principales, incluyendo los procesos de ingesta y transformación de datos.

Cuadro 4.1: Catálogo de recursos y operaciones de la interfaz de programación.

Recurso (<i>path</i>)	Método	Propósito funcional
Módulo de catálogo		
/catalogo/programas	GET	Recuperar la lista de planes de estudio existentes.
/catalogo/programas	POST	Registrar un nuevo programa académico en el sistema.
/catalogo/asignaturas	GET	Listar todas las materias disponibles en el catálogo.
/catalogo/asignaturas/{id}	PUT	Actualizar los metadatos de una asignatura específica.
Módulo de recursos		
/recursos/aulas	GET	Consultar la disponibilidad y características de las aulas.
/recursos/profesores	GET	Listar el personal docente y su estado de conciliación.
/recursos/profesores/{id}	PUT	Modificar el perfil o la disponibilidad de un docente.
Módulo de docencia		
/docencia/grupos	POST	Crear una nueva agrupación de alumnos para una materia.
/docencia/sesiones	GET	Recuperar todas las sesiones de clase planificadas.
/docencia/sesiones	POST	Asignar una nueva sesión temporal al calendario.
/docencia/horarios	GET	Obtener la planificación completa incluyendo relaciones.
Módulo de conflictos		
/conflictos/detectar	POST	Ejecutar el motor de análisis sobre un set de sesiones.
/conflictos/reglas	GET	Consultar las normativas de validación activas.
Ingesta		
/ingesta/fichas/procesar	POST	Carga y transformación de archivos de guías docentes.
/ingesta/horarios/extraer	POST	Ejecución del análisis sobre documentos de horarios.
/ingesta/horarios/confirmar	POST	Persistencia definitiva de los datos extraídos y validados.

Gestión de estados y errores

El diseño de la API utiliza los códigos de estado del protocolo HTTP para comunicar de forma semántica el resultado de cada petición. Esta estandarización permite que el *frontend*

interprete automáticamente si una operación ha tenido éxito o si requiere una acción correctiva por parte del gestor académico. Las respuestas se han categorizado en tres niveles principales:

- Operaciones exitosas: se emplean códigos como el 200 (OK) para consultas satisfactorias y el 201 (*Created*) para confirmar que un nuevo recurso, como una sesión o un programa, ha sido persistido correctamente.
- Errores de validación y cliente: el sistema utiliza el código 422 (*Unprocessable Entity*) cuando los datos enviados no cumplen con los contratos de datos establecidos. En estos casos, la respuesta incluye un objeto JSON detallando el campo exacto que ha fallado, permitiendo una retroalimentación precisa en la interfaz. Otros códigos como el 404 (*Not Found*) se reservan para el acceso a recursos inexistentes.
- Errores de servidor: el código 500 (*Internal Server Error*) se emite ante fallos imprevistos en la lógica de procesamiento, asegurando que el sistema no exponga detalles internos de la implementación por motivos de seguridad.

Seguridad y orquestación CORS

Debido a la arquitectura desacoplada del sistema, donde el *frontend* y la API pueden residir en dominios o puertos distintos, el diseño implementa una política de intercambio de recursos de origen cruzado. Esta medida de seguridad es fundamental para prevenir ataques de falsificación de peticiones, ya que permite al servidor definir explícitamente qué orígenes tienen permiso para interactuar con la lógica de negocio.

La orquestación de esta seguridad se configura a nivel de cabeceras HTTP, donde el servidor valida el origen de cada solicitud entrante. Solo aquellas peticiones provenientes de dominios autorizados y que utilicen los métodos permitidos (GET, POST, PUT, DELETE) son procesadas. Esta capa de seguridad se complementa con la validación de esquemas mencionada anteriormente, la cual asegura que el contenido de las peticiones autorizadas sea estructuralmente correcto antes de ser procesado por la lógica interna, estableciendo así un doble control de acceso y de integridad.

4.4. Diseño de la lógica de negocio

La lógica de negocio constituye el núcleo inteligente del sistema, donde se transforman los datos en bruto en información académica válida, estructurada y coherente. Esta capa es la responsable de orquestar las reglas operativas de la institución, abstrayendo la complejidad de los algoritmos de validación y los procesos de transformación de documentos para ofrecer una interfaz funcional y simplificada al gestor académico. Esta sección se centra en describir los procesos internos que permiten al sistema no solo almacenar información, sino interpretarla y validarla activamente.

Para asegurar una arquitectura robusta y mantenible, la lógica de negocio se ha estructurado en torno a flujos de trabajo especializados o *pipelines*, cada uno con una responsabilidad clara dentro del ciclo de vida del dato académico. Estos procesos abarcan desde la extracción automatizada de información contenida en documentos no estructurados hasta la evaluación multidimensional de la planificación horaria mediante un motor de detección de conflictos. De este modo, el sistema garantiza que cualquier cambio o nueva incorporación de datos pase por un estricto proceso de normalización y verificación, manteniendo la integridad de la planificación en todo momento.

4.4.1. *Pipeline* de ingesta de fichas

El catálogo académico del sistema se nutre de un proceso de ingesta automatizado que transforma las fichas docentes de las asignaturas en entidades estructuradas. El diseño de este flujo se fundamenta en el principio de que el documento PDF es la fuente de verdad absoluta; por tanto, el sistema no solo realiza una importación inicial, sino que sincroniza de forma continua el catálogo para reflejar cualquier actualización en la estructura de las asignaturas. Este enfoque elimina la necesidad de un registro manual de las asignaturas y minimiza los errores de transcripción en datos críticos como códigos oficiales o carga de créditos.

La arquitectura del sistema asume que la información contenida en las fichas académicas prevalece sobre los registros existentes en la base de datos. Para materializar esta visión, se ha diseñado una lógica de persistencia basada en la estrategia *upsert* (*update* o *insert*), la cual permite que, al procesar un documento, que el sistema identifique si la asignatura ya existe mediante su código único para actualizar sus atributos o crear un nuevo registro en caso contrario. Este diseño garantiza la idempotencia del proceso, permitiendo que un mismo archivo sea procesado múltiples veces sin generar duplicidades y asegurando que la base de datos converja siempre hacia el estado más reciente del documento oficial.

Estructura del proceso: el *pipeline* de cuatro etapas

El procesamiento de cada ficha se organiza en una secuencia de cuatro fases especializadas, donde la salida de cada etapa sirve como entrada validada para la siguiente:

1. **Fase de extracción y control de calidad:** el primer módulo se encarga de convertir el PDF nativo en texto plano. Dada la heterogeneidad de estos documentos (véase el ejemplo real en el Anexo A), se utiliza una extracción exclusivamente nativa para preservar la estructura original. Una característica distintiva de este diseño es la inclusión de un umbral de legibilidad multidimensional; el sistema evalúa métricas como la densidad de caracteres alfabéticos, la proporción de palabras cortas y la presencia de patrones académicos específicos como códigos de asignatura. Esta evaluación asigna un nivel de calidad (*excellent*, *good*, *acceptable*, *poor*) que actúa como un cortafuegos: si el documento no alcanza un *score* mínimo de viabilidad, el *pipeline* aborta la operación antes de consumir recursos en el análisis semántico.
2. **Fase de *parsing* semántico:** una vez obtenido el texto, el motor de análisis utiliza patrones de expresiones regulares y lógica posicional para identificar bloques de información. El diseño permite localizar de forma selectiva datos como el nombre de la materia, los créditos ECTS (*European credit transfer and accumulation system*) y el profesorado, transformando cadenas de texto amorfas en objetos de dominio académico.
3. **Fase de normalización:** en esta etapa, los datos extraídos se someten a un proceso de estandarización. Se eliminan ruidos tipográficos y se realiza un mapeo hacia tipos enumerados, asegurando que conceptos como el periodo lectivo o el idioma de impartición utilicen valores uniformes que puedan ser interpretados por el motor de conflictos.
4. **Fase de persistencia transaccional:** el flujo culmina con el volcado de datos en el sistema de persistencia. Esta operación, diseñada bajo los principios ACID, garantiza que la actualización de la asignatura y la vinculación con sus programas y docentes asociados se realice de forma atómica. Si ocurre un error en cualquier punto de la grabación, el sistema ejecuta un *rollback* completo para evitar estados inconsistentes en el catálogo.

Estas fases, que se detallan visualmente en el diagrama de actividad del proceso (Figura 4.3), aseguran que la información transite desde un estado no estructurado hasta una persistencia garantizada por el sistema. El flujo diseñado permite que cada etapa actúe como un filtro de calidad antes de permitir el acceso a la base de datos.

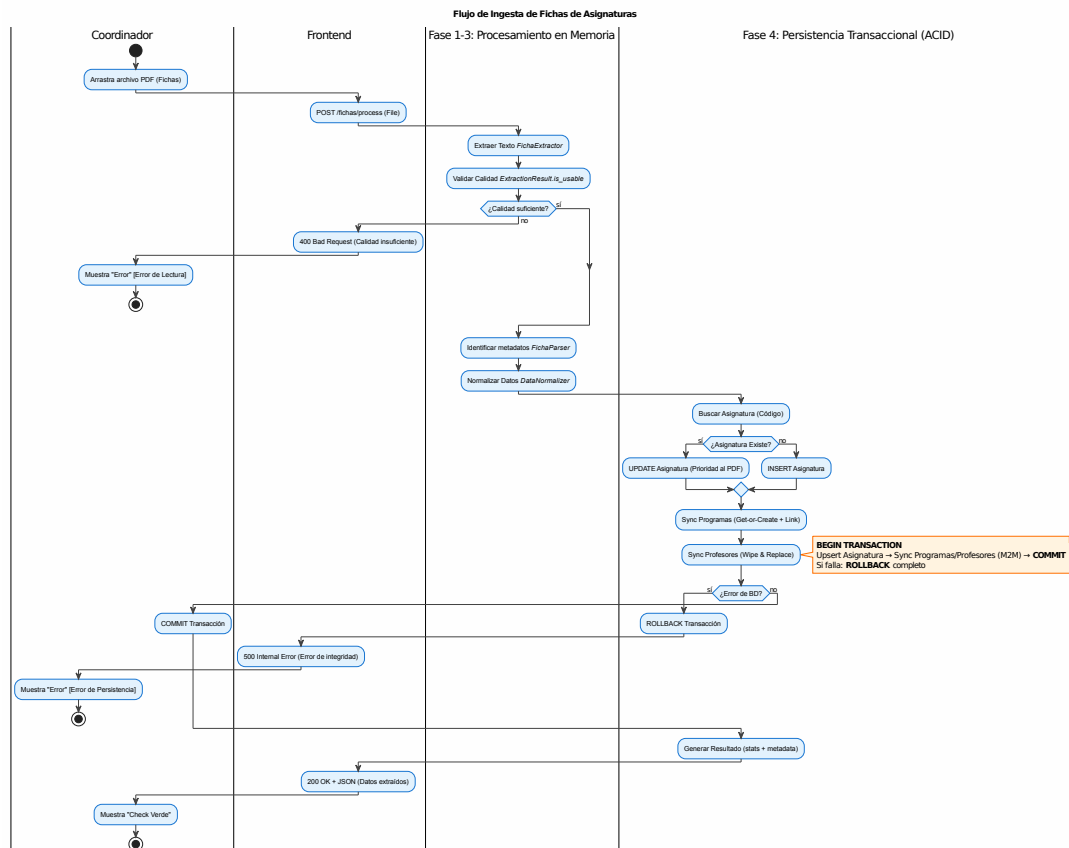


Figura 4.3: Diagrama de actividad del *pipeline* de ingestión de fichas académicas y gestión de calidad.

Estrategias de deduplicación y sincronización diferencial

Para mantener la integridad del catálogo global, el sistema implementa una lógica de gestión de entidades compartidas. En el caso de programas académicos y personal docente, el diseño emplea una estrategia de creación bajo demanda: antes de registrar un nuevo profesor o programa, se verifica su existencia en el sistema para reutilizar el registro existente, evitando la fragmentación de la información.

Un aspecto avanzado del diseño es la sincronización diferencial aplicada a las relaciones docentes. El sistema no se limita a añadir los profesores detectados en el documento, sino que realiza una comparación con los registros previos en la base de datos. Si un docente ya no figura en la ficha actualizada, el proceso elimina automáticamente dicha vinculación en el sistema. Esta lógica asegura que el catálogo de responsabilidades docentes sea siempre un reflejo fiel y actualizado de la documentación oficial, gestionando el ciclo de vida de las relaciones sin intervención humana.

4.4.2. *Pipeline* de ingestión de horarios

El segundo componente fundamental de la lógica de negocio es el proceso de ingestión de horarios, cuya finalidad es registrar en el sistema la planificación horaria del centro para habilitar las capacidades de detección de conflictos del motor analítico. Mientras que el flujo de fichas se encarga de definir la estructura y los atributos de las asignaturas en el catálogo académico, este se centra en la captura técnica de las sesiones de clase para su posterior validación. Este flujo permite que el sistema deje de ser un repositorio estático de información y se convierta en una herramienta activa de supervisión académica.

El diseño de este *pipeline* responde a la necesidad de automatizar la generación del calendario escolar partiendo de documentos que pueden variar en formato y calidad de texto. Por esta razón, la lógica de negocio incorpora mecanismos de mapeo espacial para identificar celdas en tablas y algoritmos de *fuzzy matching* para interpretar la información en entornos donde la nomenclatura de las asignaturas no es uniforme[6].

A diferencia del flujo de fichas, cuya persistencia es directa, el *pipeline* de horarios se ha diseñado para que tenga una naturaleza consultiva y desacoplada. Se ha proyectado una arquitectura de estado de borrador en memoria, donde los datos extraídos no persisten en la base de datos hasta que se produce una supervisión y validación de la interpretación automática. Este diseño asegura una separación conceptual clara entre la información bruta extraída y el horario final, permitiendo que el sistema funcione como un asistente que propone vínculos entre el documento y el catálogo, pero delegando la integridad final en la validación del gestor.

Un aspecto estratégico del diseño es su capacidad de aprendizaje supervisado. Mediante un modelo de gestión de alias, el sistema está diseñado para vincular terminologías informales con los registros oficiales del catálogo de forma dinámica. Esta retroalimentación se integra en el flujo de consolidación del horario, permitiendo que el sistema evolucione su precisión y reduzca progresivamente la necesidad de ajustes manuales en futuras iteraciones del proceso.

Estructura del proceso: el *pipeline* de tres fases

El diseño de este flujo se divide en tres etapas secuenciales que garantizan la transición desde la ambigüedad del documento original hasta la precisión requerida por el calendario académico:

1. **Extracción y análisis espacial:** el diseño de la primera etapa del *pipeline* se aleja de una extracción de texto lineal convencional para adoptar un enfoque de reconstrucción geométrica y mapeo espacial. Dado que la planificación horaria se representa habitualmente en estructuras matriciales o rejillas de tiempo, tal y como se evidencia en el Anexo B, la lógica de negocio debe ser capaz de interpretar no solo los caracteres, sino la posición relativa de los mismos dentro del lienzo del documento. Esta fase se fundamenta en los siguientes pilares de diseño:
 - Identificación de la estructura tabular: el sistema proyecta un análisis de la disposición de los datos para detectar celdas y filas, permitiendo que la información fragmentada en el PDF recupere su contexto bidimensional.
 - Mapeo de coordenadas y ejes temporales: la lógica de negocio asocia las coordenadas espaciales de cada bloque de texto con ejes lógicos predefinidos; el eje horizontal se vincula a la secuencia de los días de la semana, mientras que el eje vertical se traduce en franjas horarias. Esta asociación permite inferir la duración y el momento de cada sesión sin necesidad de que estos datos figuren explícitamente como texto junto a cada asignatura.
 - Generación de estructuras semánticas en memoria: el resultado de esta fase no es una persistencia definitiva, sino la creación de un objeto de datos de salida temporal que agrupa las sesiones detectadas. Este objeto actúa como un contenedor de borrador que preserva tanto el texto original extraído como los metadatos de posición, preparando la información para ser procesada por el motor de resolución de entidades en la siguiente fase.
2. **Resolución contextual y *fuzzy matching*:** el diseño de la segunda fase del *pipeline* actúa como el motor de interpretación semántica del sistema. Su función principal es resolver la discrepancia entre el texto extraído del documento (frecuentemente abreviado o incompleto) y las entidades oficiales almacenadas en la base de datos. Para maximizar la precisión y reducir los falsos positivos, el diseño no se basa en una búsqueda genérica, sino

en una arquitectura de resolución jerárquica y contextual. Esta fase se estructura bajo los siguientes principios de diseño:

- Resolución jerárquica orientada al contexto: el sistema está diseñado para reducir el universo de búsqueda mediante la inferencia previa del programa académico y el curso, basándose en los metadatos detectados en el encabezado del documento. Este filtrado contextual permite que, ante un término ambiguo, el sistema priorice las asignaturas que pertenecen al plan de estudios específico que se está procesando, evitando colisiones entre materias homónimas de diferentes titulaciones.
- Algoritmo de comparación difusa con umbrales de confianza: el diseño utiliza métricas de similitud textual para evaluar la coincidencia entre el texto del horario y el catálogo. El sistema asigna un nivel de confianza a cada vínculo propuesto, clasificando los resultados en diferentes estados de emparejamiento: coincidencias exactas, sugerencias automáticas de alta confianza o candidatos de baja confianza que requieren supervisión manual.
- Arquitectura de búsqueda optimizada: para garantizar un rendimiento fluido, el diseño contempla el uso de índices multidimensionales y estructuras de búsqueda en memoria que agrupan tanto los nombres oficiales como los alias previamente confirmados. Esto permite que el motor de búsqueda sea capaz de identificar una asignatura incluso cuando se utilizan nomenclaturas muy alejadas de la denominación formal, siempre que exista un registro histórico de dicha equivalencia.
- Interfaz de validación y supervisión humana: el diseño culmina con la presentación de una vista de resultados donde se contrastan los datos extraídos con las sugerencias del motor de resolución. En este punto, el sistema expone visualmente el nivel de confianza de cada emparejamiento, permitiendo al gestor validar las propuestas automáticas, corregir asignaciones de baja confianza o descartar sesiones erróneas antes de proceder a la siguiente fase. Esta etapa de control es el componente que garantiza que el proceso sea verdaderamente asistido y no puramente autónomo, asegurando que solo la información supervisada transite hacia la base de datos definitiva.

3. **Consolidación y persistencia transaccional:** para la última etapa, el enfoque de diseño debe centrarse en la integridad de la persistencia y el aprendizaje del sistema. Estos son los puntos clave que se cubrirán:

- Estrategia de limpieza selectiva (*wipe & replace*): el diseño proyecta una limpieza granular que elimina únicamente las sesiones y conflictos previos de las asignaturas presentes en el nuevo horario. Esto evita la existencia de registros obsoletos sin afectar a la planificación de materias que no forman parte del documento procesado.
- Mecanismo de aprendizaje mediante alias: se describe el diseño de la persistencia de nuevas equivalencias. Si el usuario valida una sugerencia donde el texto del documento difiere del nombre oficial, el sistema genera automáticamente un registro de alias que optimizará futuras extracciones.
- Persistencia atómica y disparo del motor analítico: la consolidación se diseña como una operación transaccional única. Una vez que las sesiones son persistidas con éxito, el diseño contempla la llamada inmediata al motor de detección de conflictos para evaluar la nueva planificación en tiempo real.

La integración de estas tres fases y la interacción entre los diferentes componentes del sistema se detallan en el diagrama de actividad integral del proceso (véase la Figura 4.4). En este esquema se observa cómo el flujo de datos transita desde la extracción geométrica hasta la consolidación atómica, resaltando el papel crítico del gestor en la validación de las sugerencias del motor de lógica difusa antes de la persistencia definitiva y el posterior análisis de colisiones.

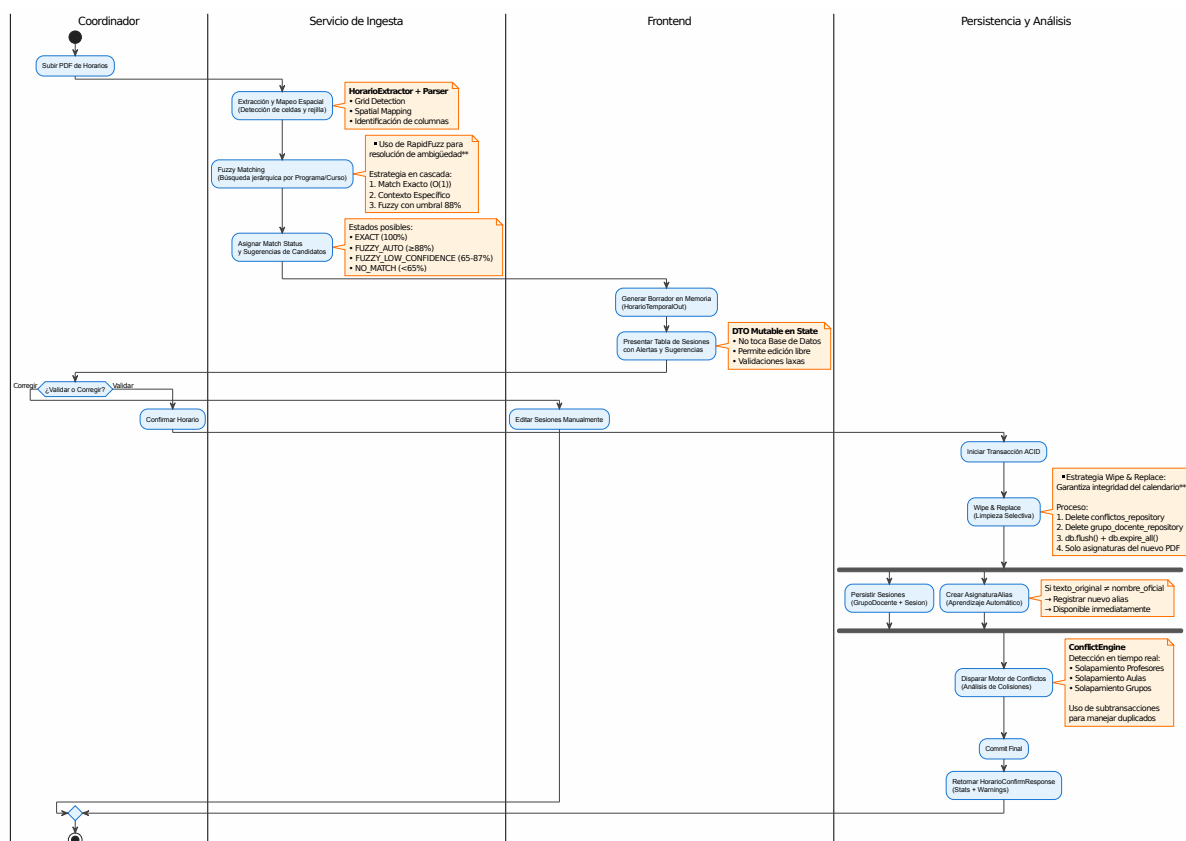


Figura 4.4: Diagrama de actividad integral del pipeline de ingesta de horarios y detección de conflictos.

Estrategias de deduplicación y sincronización diferencial de sesiones

Para mantener la integridad del calendario, el sistema aplica una variante de la estrategia de sincronización diferencial orientada a la sustitución selectiva de la planificación. Dado que un horario puede sufrir múltiples actualizaciones durante su fase de elaboración, el diseño debe garantizar que la re-ingesta de un documento no genere duplicidades de sesiones ni mantenga registros de versiones obsoletas. Esta lógica se implementa bajo los siguientes criterios de diseño:

- **Limpieza granular por asignatura:** a diferencia de un borrado masivo, el sistema realiza un rastreo de las asignaturas presentes en el nuevo documento. Antes de la persistencia, se eliminan exclusivamente las sesiones y grupos asociados a dichas materias en el catálogo actual. Este enfoque permite actualizar el horario de un grado o curso específico sin alterar el resto de la planificación global del centro.
- **Deduplicación de recursos físicos y humanos:** el sistema emplea una lógica de “creación bajo demanda” para aulas y profesores. Durante el procesamiento de las sesiones, se verifica la existencia de estos recursos en el sistema para reutilizar los registros existentes. En el caso de las aulas, si el texto del documento no coincide con el catálogo, el motor de resolución difusa propone el espacio físico más probable para facilitar el emparejamiento.
- **Consistencia de la estructura docente:** la sincronización asegura que los grupos de alumnos se vinculen de forma coherente con sus respectivas sesiones. Al aplicar la estrategia de limpieza y sustitución, el sistema regenera las relaciones entre asignaturas, grupos y aulas, garantizando que el motor analítico trabaje siempre sobre la versión más reciente y veraz de la ocupación de espacios.

4.4.3. Motor de detección de conflictos

El motor de detección de conflictos constituye el núcleo analítico de la aplicación, transformando el sistema de un simple repositorio de horarios en una herramienta inteligente de supervisión académica. Su propósito fundamental es garantizar la viabilidad operativa del calendario académico mediante la identificación automática de colisiones lógicas en el uso de recursos críticos: espacios físicos, disponibilidad docente y concurrencia estudiantil.

El diseño de este motor se aleja de una validación estática para adoptar un enfoque de análisis dinámico y transaccional. Cada modificación en la planificación horaria dispara una evaluación de integridad que garantiza que la información persistida cumpla con las restricciones de negocio en todo momento. Para lograr esto, el sistema integra una lógica de intersección temporal optimizada y una arquitectura de persistencia reflexiva que permite gestionar las relaciones de conflicto de forma eficiente.

Definición y taxonomía de conflictos

En el contexto de este sistema, se define un conflicto como la detección de una incompatibilidad temporal en la asignación de recursos necesarios para el desarrollo de la actividad académica. El motor de búsqueda no se limita a identificar solapamientos de horas, sino que valida la integridad de la planificación basándose en la disponibilidad de cuatro recursos fundamentales:

- **Solapamiento de aulas:** se identifica cuando dos sesiones diferentes coinciden en el mismo espacio físico durante una misma franja horaria. El sistema protege la exclusividad del uso del aula para evitar duplicidades de reserva.
- **Solapamiento de profesor:** ocurre cuando un mismo profesor tiene asignadas sesiones simultáneas en diferentes grupos o asignaturas. Esta restricción responde a la imposibilidad física de bilocación del personal docente.
- **Solapamiento de grupos docentes:** se detecta cuando un grupo específico de alumnos tiene programadas dos o más clases al mismo tiempo. Este conflicto es crítico, ya que invalida el itinerario formativo del estudiante al impedirle la asistencia a la totalidad de sus actividades académicas programadas.
- **Incumplimiento de la disponibilidad docente:** este conflicto no nace de una imposibilidad física inmediata en el centro actual, sino de la vulneración de las franjas horarias marcadas como restringidas por el docente. El sistema identifica si una sesión ha sido programada en un horario marcado como “no disponible”, ya sea por motivos de conciliación personal o por obligaciones docentes en otros centros o campus. Esta distinción permite que el motor de conflictos actúe no solo como un validador técnico, sino como una herramienta de gestión humana y organizativa, capaz de contemplar las restricciones externas que afectan a la realidad del personal académico.

Logica de severidad y estados

El diseño del motor clasifica las colisiones según su impacto real en la operatividad del centro, permitiendo una gestión jerarquizada de las soluciones:

- **Severidad crítica (conflictos bloqueantes):** representan situaciones que deben ser resueltas obligatoriamente para garantizar la viabilidad del horario. Incluye los solapamientos de aula y de grupo docente, así como los solapamientos de profesor en los que el docente involucrado es el único asignado a las sesiones concurrentes, haciendo físicamente imposible su asistencia.

- **Severidad no bloqueante (revisiones):** agrupa conflictos que no impiden necesariamente el desarrollo docente pero requieren supervisión. Se incluyen los incumplimientos de conciliación y aquellos solapamientos donde existen otros docentes en la ficha de la asignatura con competencia para impartir esa sesión específica (según el tipo de actividad, grupo o naturaleza de la sesión) que podrían relevar al profesor afectado.
- **Severidad leve:** definida como una categoría para futuras extensiones informativas del sistema, aunque actualmente no se asocia a ninguna regla de negocio activa.

En cuanto al ciclo de vida, el sistema apuesta por la simplicidad operativa, reduciendo el flujo a dos estados posibles: por revisar y solucionado. Un conflicto nace en estado pendiente tras su detección automática y solo transita al estado solucionado cuando el gestor realiza el ajuste correspondiente en la planificación o valida la situación, asegurando así que ninguna colisión pase desapercibida durante la fase de diseño del calendario.

Algoritmo de detección: intersección de intervalos

El diseño del algoritmo se basa en un principio de geometría temporal, modelando las sesiones como intervalos definidos por $[t_{inicio}, t_{fin}]$.

Para optimizar el cómputo, el sistema aplica una lógica de exclusión de conjuntos disjuntos. Se asume la existencia de un conflicto a menos que se cumpla una de las condiciones de separación total: $SesinA.fin \leq SesinB.inicio$ o $SesinB.fin \leq SesinA.inicio$.

Si ninguna se cumple, existe intersección. El uso de operadores relacionales estrictos permite que las sesiones consecutivas (por ejemplo, fin a las 10:00 e inicio a las 10:00) no generen colisión.

Para optimizar el rendimiento y trabajar sobre un volumen de datos manejable, el diseño no aplica el algoritmo sobre el total de sesiones, sino que incorpora un filtrado multicriterio en cascada que reduce el valor de N (número de sesiones comparadas) antes de la validación:

- Segregación temporal: solo se comparan sesiones del mismo día de la semana y periodo lectivo.
- Filtrado por recurso: el motor reduce el universo de búsqueda a sesiones que compartan el mismo ID de aula, profesor o grupo.
- *Hash* de integridad simétrico: se genera una clave única para cada par de sesiones, garantizando que si se detecta el conflicto $A \rightarrow B$, el sistema no procese de nuevo la relación inversa $B \rightarrow A$.

Persistencia y sincronización de resultados

El motor de detección opera mediante un patrón de sincronización transaccional que garantiza la coherencia del calendario ante cualquier edición. El proceso se articula en tres fases críticas:

1. Invalidación selectiva (*wipe*): ante cualquier cambio en una sesión, el sistema identifica y elimina quirúrgicamente todos sus registros de conflicto previos. Esta limpieza evita la persistencia de colisiones obsoletas o “fantasmas” derivadas de estados anteriores.
2. Recálculo y análisis comparativo: tras la limpieza, el motor ejecuta la lógica de detección sobre la nueva versión de los datos, aplicando filtros de segregación temporal y espacial para comparar la sesión únicamente con el subconjunto de recursos relevante (aula, profesor, grupo y conciliación).
3. Persistencia atómica y reflexiva: los hallazgos se insertan en una tabla de relación reflexiva que permite la navegación bidireccional entre sesiones afectadas. Toda la operación se realiza bajo una única transacción de base de datos; cualquier fallo durante el proceso dispara un *rollback* automático para mantener la integridad del estado anterior.

4.4.4. Dinámica de gestión de horarios

La efectividad del motor analítico descrito en los apartados anteriores depende directamente de una interfaz de usuario que sea capaz de presentar información compleja de forma digerible y accionable. La dinámica de gestión en esta plataforma se ha diseñado bajo el paradigma de interfaz asistida, donde el sistema no solo actúa como un repositorio de datos, sino como un colaborador que guía al gestor académico en la toma de decisiones. El diseño de la interfaz de usuario se articula sobre tres pilares fundamentales:

1. **Visibilidad del estado del sistema:** mediante el uso de una arquitectura de estados claros (“Por Confirmar”, “Procesando”, “Confirmado”), el usuario conoce en todo momento la validez y la persistencia de sus cambios.
2. **Prevención y resolución de errores:** la interfaz integra visualmente las colisiones detectadas por el motor de conflictos, permitiendo una transición fluida entre la detección del problema y su resolución manual o sugerida.
3. **Eficiencia operativa:** a través de flujos de trabajo especializados para la ingesta masiva de documentos y la edición interactiva mediante rejillas de arrastrar y soltar (*drag and drop*).

A continuación, se describen los módulos principales de la interfaz, analizando cómo su diseño y jerarquía visual soportan los procesos de negocio de la institución.

Módulos de ingesta de documentos PDF

El proceso de alimentación del sistema comienza en los módulos de ingesta, diseñados específicamente para transformar la carga de archivos PDF en un flujo de trabajo sencillo y controlado. Para ello, el sistema dispone de dos interfaces de carga independientes: una dedicada exclusivamente a la subida de fichas docentes, que activa el motor de extracción de la estructura académica, y otra para la subida de horarios, que inicia el proceso de mapeo espacial y detección de sesiones. Ambas comparten un diseño basado en la simplicidad operativa y la transparencia en el procesamiento de datos.

La interfaz centraliza la subida de documentos en una zona de interacción denominada *upload dropzone*. Este componente utiliza el estándar de *drag and drop*, permitiendo al usuario interactuar de forma natural con el sistema de archivos del sistema operativo. Visualmente, la jerarquía se apoya en una tarjeta contenedora con iconos descriptivos y texto instructivo, lo que minimiza la curva de aprendizaje y reduce la carga cognitiva del gestor al realizar tareas de subida masiva.

Dado que los motores de extracción no son procesos instantáneos, el diseño de la interfaz pone especial énfasis en la visibilidad del estado del sistema:

- **Gestión de estados asíncronos:** a través de un almacén de estados (*store*) dedicado, la interfaz rastrea el ciclo de vida de cada archivo, transitando por los estados de subida (*uploading*), procesamiento en servidor (*processing*) y finalización exitosa o error.
- **Indicadores de progreso dinámicos:** para mitigar la incertidumbre del usuario, se emplean barras de progreso y componentes de carga (*spinners*) que ofrecen retroalimentación visual constante sobre el avance de la tarea en segundo plano.

La robustez del módulo de ingesta se completa con un sistema de notificaciones no intrusivas (*toasts*). Ante cualquier incidencia técnica —como un formato de archivo no válido o una interrupción en la red—, el sistema despliega alertas informativas que detallan el error sin interrumpir el flujo de trabajo global. Este diseño permite que el usuario pueda gestionar múltiples archivos simultáneamente, manteniendo siempre el control sobre qué documentos han sido integrados correctamente en el ecosistema de la aplicación.

Entorno de revisión y mapeo

Tras el procesamiento de los archivos, el sistema redirige al usuario al entorno de revisión y mapeo. Es importante destacar que esta interfaz pertenece exclusivamente al flujo de ingesta de horarios, ya que la naturaleza crítica de estos datos y la variabilidad en la calidad de los documentos PDF originales exigían un paso intermedio de validación.

Debido a que el motor de extracción puede presentar imprecisiones causadas por la baja resolución o el formato heterogéneo de los PDF de horarios, esta pantalla se diseñó como un filtro de integridad. Su objetivo es permitir que el gestor actúe como supervisor del proceso, verificando la información extraída antes de que esta sea procesada por el motor de detección de conflictos o persistida de forma definitiva.

El diseño se organiza mediante una *split view* que permite supervisar la información de forma integral (véase la Figura 4.5). En el área principal se despliega una tabla con las sesiones detectadas (asignatura, aula, grupo y franja horaria), apoyada por los siguientes elementos:

- **Indicadores de fiabilidad:** los datos se marcan visualmente según el nivel de certidumbre del extractor. Aquellos campos cuya extracción no ha sido totalmente nítida se resaltan para captar la atención inmediata del usuario.
- **Grid de previsualización rápida:** se ofrece una representación visual simplificada del horario resultante para que el usuario pueda validar la coherencia estructural de la ingesta de un vistazo.

El la parte superior del *grid* se encuentra el panel de sugerencias, alimentado por el algoritmo de *fuzzy matching*. Dado que los términos en los PDF suelen contener abreviaturas o errores tipográficos, este panel propone los vínculos más probables con las entidades reales de la base de datos:

- **Tarjetas de emparejamiento:** cada sugerencia muestra el término original detectado, la propuesta del sistema y el porcentaje de similitud calculado.
- **Acciones de resolución:** el usuario puede aceptar la propuesta con un solo clic, rechazarla o realizar una búsqueda manual si ninguna sugerencia es correcta.

Este entorno garantiza que solo la información fidedigna y correctamente mapeada se consolide en el sistema, asegurando la precisión del análisis posterior.

Revisión de horario
Verifica los datos extraídos antes de confirmar.

Resumen

Plan Detectado: **Grado En Matemáticas** | Período: **Primer Cuatrimestre** | Total: **6 cursos - 74 sesiones**

Cursos: **1º** **2º** **3º** **4º - MATEMÁTICA PURA Y APLICADA** **4º - INFORMÁTICA** **4º - ECONOMÍA Y FINANZAS**

Errores detectados
2 errores bloqueantes que impiden guardar. Además, hay 10 sugerencias automáticas disponibles para revisión.

Hora	L	M	X	J	V
08:30	Álgebra Lineal I AULA 1	Introducción Al Lenguaje Mate... AULA 1	Álgebra Lineal I AULA 1		Álgebra Lineal I AULA 1
09:30	Introducción Al Lenguaje Mate... AULA 1	Álgebra Lineal I AULA 1	Introducción Al Lenguaje Mate... AULA 1	Cálculo Diferencial AULA 1	Introducción Al Lenguaje Mate... AULA 1
10:30	Cálculo Diferencial Grupo Grupo 1	Física Básica Experimental Laboratorio ATC	Cálculo Diferencial AULA 1	Física Básica Experimental AULA 1	Física Básica Experimental AULA 1
11:30					
12:30	Física Básica Experimental Laboratorio ATC		Física Básica Experimental Laboratorio ATC		
13:30		Física Básica Experimental Laboratorio ATC		AULA 1 ODS	AULA 1 ODS
14:30					

Figura 4.5: Interfaz de revisión de horarios tras el proceso de ingesta. Se observan los metadatos del horario (curso y periodo), la lista de sugerencias del algoritmo *fuzzy matching* y el *grid* con las sesiones.

Creación y edición manual de horarios

Aunque el sistema está optimizado para la ingesta documental, dispone de un flujo completo para la creación de calendarios desde cero y la edición interactiva de los existentes. Este módulo está diseñado para ofrecer una experiencia de usuario fluida, reduciendo la posibilidad de introducir errores manuales mediante el uso de asistentes y componentes visuales reactivos.

Como paso previo al lienzo de edición, el diseño incorpora un asistente de configuración que actúa como filtro de integridad inicial. Este componente asegura que cualquier operación posterior se realice dentro de un marco académico coherente. El usuario define el curso, grado y periodo lectivo, permitiendo que el sistema precargue únicamente las asignaturas pertinentes a ese contexto. Al establecer estas restricciones de entrada, se imposibilita técnicamente la asignación accidental de docentes o espacios que no pertenezcan al plan de estudios actual.

El núcleo de la gestión manual se sitúa en una rejilla temporal que funciona como un lienzo dinámico. Las decisiones de diseño en este componente buscan emular la agilidad de las herramientas de planificación modernas:

- **Manipulación directa de sesiones:** el diseño contempla el uso de metáforas visuales de “arrastrar y soltar” para facilitar la reubicación de clases. Esta decisión busca que el usuario pueda explorar cambios en la planificación de forma táctica y rápida.
- **Previsualización de impacto:** se proyecta un sistema de *ghost preview* (previsualización traslúcida) que indica el área de destino antes de confirmar la acción, permitiendo al usuario anticipar visualmente si el nuevo hueco horario es suficiente para la duración de la sesión.
- **Acceso contextual a metadatos:** mediante la interacción con las celdas, se despliegan formularios modales que permiten modificar los atributos específicos de la sesión (como las horas de inicio y fin y el aula asignada) sin perder la visión panorámica del horario global.

Para fomentar la experimentación, el diseño separa la edición de la confirmación definitiva. Se implementa un estado de “edición en curso” que permite al gestor realizar múltiples movimientos y resolver conflictos de forma iterativa. Solo cuando el usuario considera que la propuesta es válida, se inicia el flujo de sincronización con el motor analítico, garantizando que el servidor no procese estados intermedios incompletos.

Centro de control y resolución de conflictos

El centro de control representa el componente de la interfaz diseñado para cerrar el ciclo de retroalimentación entre el motor de detección y el usuario. Su diseño se aleja de una simple lista de errores para convertirse en un panel de auditoría de calidad que guía al gestor en la depuración del calendario académico.

La interfaz implementa una conexión estrecha entre el listado de incidencias y la visualización temporal, permitiendo una inspección detallada de cada colisión:

- **Panel de auditoría de conflictos:** se proyecta una vista centralizada donde cada conflicto se presenta como una entidad expandible. Al desglosar un error, el sistema muestra tarjetas comparativas de las sesiones implicadas, detallando asignatura, docente, aula y grupo afectado para facilitar el diagnóstico inmediato del problema (véase la Figura 4.6).
- **Sincronización de enfoque visual:** el diseño contempla que la detección de un conflicto active indicadores visuales en la rejilla horaria. Las sesiones en conflicto se resaltan con bordes rojos y alertas contextuales, permitiendo al usuario identificar el impacto del error sobre la estructura global del horario.

Para facilitar la toma de decisiones basada en prioridades, el diseño organiza la información mediante una jerarquía clara de severidad:

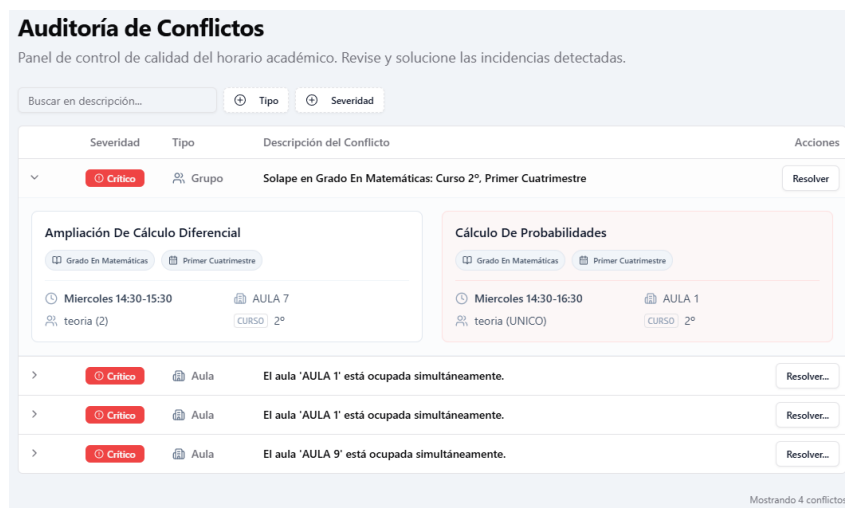


Figura 4.6: Interfaz de auditoría de conflictos. Se observa el desglose de un conflicto crítico de grupo, mostrando la comparación detallada entre las sesiones enfrentadas y sus metadatos asociados.

- **Codificación por severidad:** los conflictos se categorizan mediante *badges* de color. Los errores de tipo “Crítico” (como solapes de aula o grupo) bloquean la validación del horario, mientras que las advertencias sugieren revisiones preventivas.
- **Resolución integrada:** cada conflicto en la lista dispone de un acceso directo de “Resolver”, que vincula al usuario con las herramientas de edición manual para subsanar la incidencia sin perder el contexto de la auditoría (como se ilustra en la Figura 4.7).

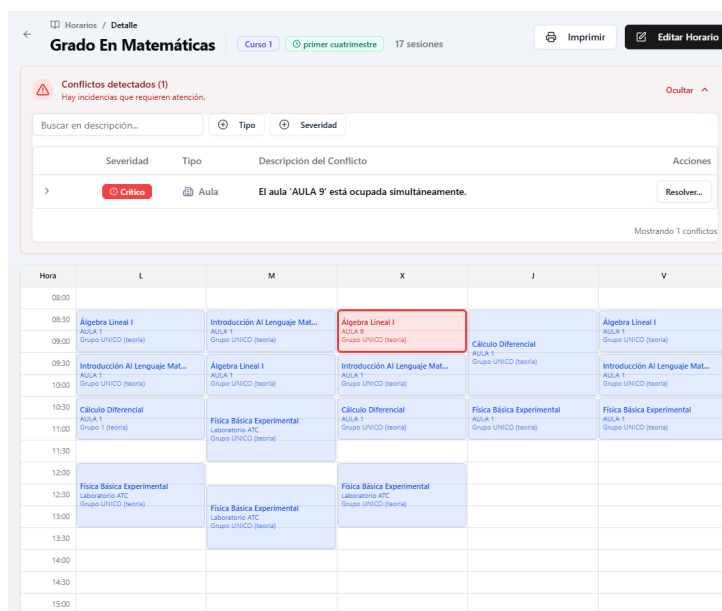


Figura 4.7: Vista integrada de gestión de conflictos. La interfaz combina el panel superior de auditoría con la rejilla horaria inferior, donde la sesión conflictiva se resalta automáticamente para facilitar su localización y edición.

El diseño de este centro de control asegura que el gestor académico disponga de una visión panorámica de la integridad del horario, transformando la resolución de conflictos en un proceso iterativo, visual y altamente eficiente.

Capítulo 5

Implementación del sistema

Tras haber definido la arquitectura lógica, los modelos de datos y los contratos de interfaz en el capítulo anterior, ahora se aborda la materialización técnica de la solución. Este capítulo describe el proceso de ingeniería que ha permitido transformar los diagramas estructurales y de comportamiento definidos en la fase de diseño en un sistema de software operativo, robusto y desplegable.

El desarrollo se ha regido por la premisa de mantener una fidelidad estricta a la separación de responsabilidades planteada en la arquitectura de software. De este modo, la implementación se ha ejecutado en dos entornos tecnológicos diferenciados pero sincronizados: un servidor de lógica de negocio construido sobre el ecosistema de Python, encargado del procesamiento intensivo de datos y la validación de reglas académicas; y una interfaz de cliente desarrollada en TypeScript y React, orientada a la interactividad y la gestión del estado visual.

A lo largo de las siguientes secciones, se detalla cómo se han traducido las abstracciones teóricas en código ejecutable. Se abordará desde la configuración física del entorno y la estructura de directorios, pasando por la programación de los algoritmos de extracción y detección de conflictos, hasta la construcción de los componentes visuales y la capa de persistencia. El objetivo es demostrar la viabilidad técnica de la propuesta y justificar las decisiones de implementación adoptadas para satisfacer los requisitos de rendimiento, escalabilidad e integridad transaccional establecidos.

5.1. Estructura y configuración

La implementación del sistema se ha organizado bajo una estrategia de repositorio unificado, centralizando tanto la lógica del servidor como la interfaz de cliente en una misma estructura de directorios. Esta decisión operativa simplifica la gestión del control de versiones y garantiza que los contratos de datos compartidos entre el *backend* y el *frontend* se mantengan sincronizados durante el ciclo de vida del desarrollo. A continuación, se detalla la topología física de los archivos y los mecanismos de configuración que orquestan el arranque de la aplicación.

5.1.1. Organización física del repositorio

La arquitectura de carpetas se ha diseñado para reflejar la separación de responsabilidades definida en la fase de diseño, evitando el acoplamiento entre la capa de presentación y la lógica de negocio. En el nivel raíz, el proyecto segrega claramente los entornos de ejecución, aislando las dependencias de Python y Node.js en sus respectivos directorios de trabajo.

Estructura del *backend*

Para el servidor, desarrollado sobre el ecosistema Python, se ha descartado una organización tradicional basada puramente en capas técnicas (controladores, modelos, vistas) en favor de

una arquitectura orientada al dominio. El código fuente se distribuye en módulos funcionales independientes (*catalogo*, *docencia*, *recursos*), donde cada directorio encapsula sus propias rutas de API, servicios de negocio y esquemas de validación.

Esta estructura se complementa con un núcleo transversal que alberga los componentes agnósticos al dominio, tales como el motor de algoritmos de detección de conflictos y los procesadores de extracción de documentos PDF. De este modo, la lógica algorítmica compleja se mantiene desacoplada de los controladores HTTP, facilitando su testeo unitario y reutilización.

Estructura del *frontend*

La interfaz de usuario se ha implementado siguiendo las convenciones de *App Router* del *framework* Next.js. La organización de archivos prioriza el principio de proximidad (*colocation*), ubicando los componentes visuales y las utilidades específicas cerca de las rutas que los consumen, en lugar de dispersarlos en carpetas genéricas. Asimismo, se ha establecido una biblioteca de componentes compartidos para elementos de interfaz atómicos (botones, paneles, formularios), asegurando la consistencia visual en toda la aplicación mediante la reutilización de código.

5.1.2. Gestión de la configuración y entornos

Para garantizar la seguridad y la portabilidad del sistema entre entornos de desarrollo y producción, se ha implementado una estricta separación entre el código y la configuración. Se han eliminado las credenciales y parámetros variables del código fuente, delegando su gestión a variables de entorno cargadas dinámicamente.

En el *backend*, la gestión de la configuración utiliza la librería *pydantic-settings*, especializada en la validación robusta de entornos. En lugar de acceder directamente a las variables del sistema operativo, esta herramienta permite definir una clase de configuración (heredada de *BaseSettings*) que actúa como un esquema tipado. Esto valida automáticamente la presencia y el tipo de dato de parámetros críticos antes del arranque; por ejemplo, asegura que la cadena de conexión a la base de datos sea un *string* válido o que los orígenes permitidos para CORS se procesen como una lista de cadenas. Este componente se instancia como un *singleton*¹ inmutable y cacheado, inyectándose posteriormente como dependencia en los servicios que requieren acceso a dichas variables.

Por su parte, el *frontend* centraliza la configuración de la comunicación HTTP mediante un cliente de API tipado. Este cliente consume las variables de entorno expuestas en tiempo de construcción para determinar la ubicación del servidor, permitiendo que la aplicación web se adapte automáticamente al dominio donde sea desplegada sin requerir recompilación.

5.1.3. Inicialización y *middleware*

El punto de entrada del sistema se ha diseñado para gestionar el ciclo de vida de la aplicación de forma controlada. El arranque del servidor se orquesta mediante un gestor de contexto asíncrono (*lifespan*), el cual asegura que los recursos críticos, como el *pool* de conexiones a la base de datos y los directorios temporales para la carga de archivos, estén correctamente inicializados antes de aceptar la primera petición HTTP.

Simultáneamente, se ha configurado una capa de *middleware* para interceptar y procesar todas las solicitudes entrantes. Destaca especialmente la configuración dinámica de CORS, la cual lee la lista blanca de orígenes permitidos desde la configuración del entorno. Esto permite al servidor rechazar automáticamente cualquier petición proveniente de dominios no autorizados, estableciendo un perímetro de seguridad efectivo a nivel de aplicación antes de que el tráfico alcance la lógica de negocio.

¹Patrón de diseño creacional que garantiza que una clase tenga una única instancia durante todo el ciclo de vida de la aplicación y proporciona un punto de acceso global a ella.

5.2. Capa de persistencia y acceso a datos

La implementación de la capa de persistencia constituye el cimiento sobre el que se garantiza la integridad de la información académica. Siguiendo el diseño relacional definido previamente, se ha construido un sistema de acceso a datos robusto utilizando la librería SQLAlchemy como motor de ORM. Esta elección tecnológica permite abstraer las diferencias entre los motores de base de datos utilizados en desarrollo y producción, manteniendo al mismo tiempo un control estricto sobre las transacciones y las relaciones entre entidades.

A continuación, se describen los componentes técnicos que articulan la comunicación entre la lógica de negocio y el almacenamiento físico.

5.2.1. Configuración del motor y conexiones

La persistencia de datos se fundamenta en el motor SQLite, seleccionado por su portabilidad y la ausencia de dependencias de servidor externas, lo cual simplifica el despliegue del prototipo. La instanciación del motor (*engine*) se gestiona mediante un patrón de fábrica que implementa una configuración técnica específica para garantizar la compatibilidad con la arquitectura asíncrona de FastAPI:

- **Gestión de hilos en entorno asíncrono:** por defecto, SQLite impone restricciones de acceso monohilo. Para superar esta limitación, se ha configurado el motor con el parámetro `connect_args={check_same_thread: False}`. Esta directiva es crítica, ya que permite que los objetos de conexión creados en un hilo puedan ser utilizados en otros, soportando así el *thread pool* que utiliza FastAPI para manejar las peticiones concurrentes sin bloquear el sistema.
- **Resolución de rutas:** se ha implementado una lógica de normalización de rutas que transforma la ubicación relativa del archivo de base de datos en una ruta absoluta del sistema operativo. Esto asegura la consistencia en el acceso al fichero `.db` independientemente del directorio de trabajo desde el cual se invoque el proceso del servidor.

5.2.2. Mapeo objeto-relacional

La definición de las entidades se ha realizado utilizando el sistema declarativo de SQLAlchemy. Todas las clases del modelo heredan de una base común (`declarative_base`), lo que permite centralizar la gestión de metadatos y simplificar las migraciones de esquema.

La implementación de las relaciones entre tablas ha puesto un énfasis especial en la integridad referencial automática. En el código, esto se traduce en el uso extensivo de directivas de cascada. Para las relaciones padre-hijo fuertes, como la existente entre una sesión y su lista de profesores asignados, se ha configurado la opción `cascade=all, delete-orphan`. Esta instrucción garantiza que, si una sesión es eliminada o desvinculada, el sistema limpie automáticamente los registros intermedios huérfanos, evitando la acumulación de basura en la base de datos sin necesidad de intervención manual. Asimismo, se han definido propiedades de navegación bidireccional (`back_populates`) que permiten recorrer el grafo de objetos (por ejemplo, desde un aula hacia todas sus sesiones) de manera intuitiva dentro de la lógica de negocio.

5.2.3. Abstracción mediante repositorios

Para desacoplar la lógica de negocio de las consultas SQL directas, se ha implementado el “patrón repositorio”. A diferencia de implementaciones genéricas, se ha optado por repositorios especializados por dominio (por ejemplo, `AulaRepository`, `SesionRepository`), lo que permite optimizar las consultas según la naturaleza de cada entidad.

Un aspecto crítico de esta implementación es la separación de responsabilidades en la persistencia. Los métodos de escritura de los repositorios (`create`, `update`) han sido diseñados para ejecutar operaciones de preparación (`flush`) sobre la sesión de base de datos, pero explícitamente no ejecutan la confirmación definitiva (`commit`). Esta decisión de diseño es fundamental para permitir que la capa superior de servicios pueda orquestar múltiples operaciones de repositorios distintos dentro de una única transacción atómica.

5.2.4. Gestión transaccional y garantía ACID

La garantía de las propiedades ACID se logra mediante un estricto control del ciclo de vida de la sesión de base de datos, orquestado a través del sistema de inyección de dependencias del *framework*.

1. **Patrón *Session-per-Request***: se ha implementado un generador de sesiones que asigna una conexión única y aislada a cada petición HTTP entrante. Este componente asegura que, al finalizar la petición, la conexión se cierre correctamente, ejecutando un *rollback* automático de seguridad en caso de que se haya producido cualquier excepción no controlada durante el proceso.
2. **Transaccionalidad en servicios**: la responsabilidad de confirmar los cambios recae exclusivamente en la capa de servicios. Un servicio de negocio puede invocar múltiples métodos de repositorios (por ejemplo, crear una sesión, asignar profesores y registrar un conflicto), y solo cuando todas las operaciones han tenido éxito, ejecuta un único `db.commit()`. Esto garantiza la atomicidad: o se guardan todos los cambios, o no se guarda ninguno.
3. **Simulaciones y *savepoints***: para funcionalidades avanzadas como la “Simulación de Horarios”, donde el usuario desea ver el impacto de un cambio sin guardarlo, se ha implementado el uso de transacciones anidadas (`db.begin_nested()`). Esta técnica crea puntos de guardado (*savepoints*) que permiten al motor de conflictos evaluar los datos “sucios” en memoria y devolver los resultados al usuario, forzando posteriormente un *rollback* controlado para descartar los cambios temporales y devolver la base de datos a su estado original inmaculado.

5.3. Implementación de la lógica de negocio y API

La capa de lógica de negocio constituye el núcleo inteligente del sistema, donde convergen las peticiones de la interfaz de usuario, los procesos de extracción de PDFs y los algoritmos de validación académica. Esta sección detalla la implementación de los cuatro motores principales que componen el *backend*: la API, los flujos de ingesta de fichas y de horarios y el motor de detección de conflictos.

5.3.1. API REST y validación de datos

La exposición de la lógica de negocio se realiza a través del *framework* FastAPI, apoyándose intensivamente en la librería Pydantic para garantizar la integridad de los datos en las fronteras del sistema. A diferencia de un uso convencional donde los esquemas solo validan tipos primitivos, la implementación aprovecha el motor de validación para realizar transformaciones complejas antes de que los datos lleguen a los servicios.

Se ha configurado el modelo de validación utilizando `ConfigDict` y validadores de campo (`field_validator`) que operan en modo `before`. Esto permite interceptar los objetos ORM crudos provenientes de la base de datos y enriquecerlos dinámicamente durante el proceso de serialización. Un ejemplo crítico implementado es la transformación de las sesiones horarias: el sistema no se limita a devolver identificadores numéricos, sino que resuelve la jerarquía completa

de relaciones (sesión → grupo → asignatura → programa) para construir una representación textual legible (por ejemplo, “Grado en Matemáticas - 2º Curso”) que el cliente puede consumir directamente sin necesidad de realizar múltiples peticiones encadenadas.

Para la serialización de respuestas, se ha habilitado el modo `from_attributes=True` (equivalente al antiguo ORM *mode*), lo que permite un mapeo directo y eficiente entre las entidades de SQLAlchemy y los contratos JSON definidos en la API.

5.3.2. Implementación del flujo de ingesta de fichas

El procesamiento de las guías docentes se ha arquitecturado como un *pipeline* secuencial compuesto por tres componentes especializados, cada uno con una responsabilidad única en la transformación del documento PDF a entidades de dominio.

1. **Extractor de texto nativo (FichaExtractor):** este componente actúa como la capa de abstracción sobre el archivo binario. Se ha implementado utilizando la librería `pypdf`², seleccionada por su eficiencia en la extracción de texto embebido (*native text*) sin depender de motores de reconocimiento óptico (OCR). Su función se limita a iterar sobre las páginas del documento y concatenar el contenido textual en un flujo continuo (*raw string*), eliminando metadatos irrelevantes y gestionando la descryptación automática si el archivo lo requiere.
2. **Motor de análisis semántico (FichaParser):** el núcleo de interpretación recibe el texto crudo y aplica una batería de expresiones regulares (*regex*) diseñadas para localizar bloques de información específica. La implementación de estos patrones destaca por dos características técnicas:
 - Flexibilidad de cabeceras: se utilizan grupos de no-captura para tolerar variaciones tipográficas (como “Créditos” vs “Credits ECTS”), asegurando la detección independientemente del formato de la ficha.
 - Búsqueda contextual (*lookahead*): para evitar que la extracción de un campo invada el siguiente, se emplean aserciones de búsqueda anticipada que delimitan el final de una captura basándose en la presencia de la siguiente palabra clave lógica (por ejemplo, detener la lectura del “Nombre” cuando se detecta el inicio de “Créditos”).
 - *Parsing* estructural: para listas complejas como el profesorado, el *parser* cambia de una estrategia de búsqueda global a un análisis línea a línea, identificando viñetas y separando mediante lógica posicional los nombres de los docentes de sus roles académicos.
3. **Normalizador de datos:** la última etapa del flujo se encarga de la higienización y tipado de los datos extraídos. Este módulo transforma las cadenas de texto en tipos nativos de Python (conversión de decimales con coma a `float` para los créditos, limpieza de espacios en blanco redundantes) y mapea los valores categóricos (como “Formación Básica” u “Obligatoria”) a los enumerados oficiales del dominio, garantizando que la información persistida cumpla con la integridad del esquema de base de datos.

5.3.3. Implementación del flujo de ingesta de horarios

A diferencia de la linealidad de las guías docentes, la información de los horarios reside en su estructura bidimensional: la posición de un texto determina si corresponde a un “lunes a las 9:00” o a un “viernes a las 14:00”. Para abordar este desafío de reconstrucción espacial, el flujo se ha implementado mediante tres componentes especializados que transforman la geometría visual en semántica académica.

²PyPDF: <https://pypdf.readthedocs.io/en/stable/>

1. **Motor de extracción espacial (pdfplumber):** dado que la estructura tabular de los horarios depende estrictamente de las coordenadas, se substituyó `pypdf` por la librería `pdfplumber` para esta etapa específica. Esta herramienta permite acceder a los metadatos geométricos de cada carácter. La implementación extrae no solo el contenido textual, sino las cajas delimitadoras (*bounding boxes*) definidas por las coordenadas $(x_0, top, x_1, bottom)$. Esta precisión milimétrica es el requisito indispensable para diferenciar columnas visualmente adyacentes pero lógicamente distintas.
2. **Reconstructor geométrico (Gridmaster):** sobre los datos vectoriales brutos opera un algoritmo propio diseñado para inferir la estructura de la rejilla, independientemente de si el documento PDF contiene líneas divisorias visibles o no. Su lógica se basa en un agrupamiento por densidad:
 - *Clustering* de alineaciones: el algoritmo agrupa las coordenadas X e Y de todas las palabras detectadas. Utilizando una tolerancia en píxeles previamente configurada, identifica cúmulos de alta densidad que revelan los bordes invisibles de las filas y columnas.
 - Mapeo lógico: una vez inferidas las fronteras de la tabla, se aplica una búsqueda binaria para asignar cada palabra a una celda matricial (*Fila, Columna*), reconstruyendo en memoria una representación fiel de la distribución temporal original.
3. **Resolutor de entidades (RapidFuzz³):** el último componente aborda el problema de la discrepancia semántica: los horarios suelen utilizar abreviaturas (por ejemplo, “Mat. II”) que no coinciden con el catálogo oficial. Para vincular estos términos, se ha integrado `RapidFuzz`, una implementación optimizada en C++ de algoritmos de distancia de Levenshtein⁴. La estrategia de emparejamiento no es una búsqueda plana, sino un proceso en cascada diseñado para minimizar falsos positivos:
 - a) Búsqueda contextual: el sistema restringe primero el universo de comparación a las asignaturas del curso y programa detectados en la cabecera del documento.
 - b) *Scoring* independiente del orden: se utiliza el algoritmo `token_set_ratio`, que permite identificar equivalencias incluso si el orden de las palabras varía.
 - c) *Fallback* global: solo si el contexto no arroja resultados con una confianza superior al 85 %, se expande la búsqueda al catálogo completo de la base de datos.

5.3.4. Motor de detección de conflictos

El núcleo analítico valida la planificación temporal mediante la fórmula de intersección de intervalos: dos eventos A y B colisionan si $\max(Inicio_A, Inicio_B) < \min(Fin_A, Fin_B)$.

Aunque el diseño algorítmico admite la gestión de eventos de fecha única, la implementación actual prioriza la estabilidad, focalizando su alcance en los horarios regulares del cuatrimestre. Por ello, la detección se limita a colisiones entre sesiones semanales periódicas, manteniendo la comparación heterogénea como una funcionalidad latente.

Para reducir la complejidad de $O(N^2)$ a $O(N \times M)$, se emplea una estrategia de indexación por recurso. El sistema agrupa las sesiones en tablas *hash* (profesor, aula, grupo) antes de comparar. De este modo, en lugar de contrastar cada sesión contra la totalidad del horario (N), solo se verifica contra el subconjunto (M) que comparte el mismo recurso, permitiendo validar miles de registros en menos de dos segundos.

³RapidFuzz: *Fuzzy string matching in Python*. <https://github.com/seatgeek/thefuzz>

⁴Métrica que cuantifica la diferencia entre dos secuencias de caracteres calculando el número mínimo de ediciones de un solo carácter (inserciones, eliminaciones o sustituciones) necesarias para transformar una en la otra.

5.4. Implementación de la interfaz de usuario

La interfaz de usuario representa la capa tangible del sistema, responsable de traducir la complejidad algorítmica del *backend* en una experiencia de gestión fluida y comprensible. El desarrollo del cliente web se ha regido por principios de diseño atómico y reactividad inmediata, priorizando el rendimiento en la manipulación del DOM (*document object model*) para soportar la interactividad intensiva requerida por la rejilla de horarios.

A continuación, se detalla la arquitectura del *frontend* y la implementación técnica de sus componentes críticos.

5.4.1. Arquitectura de componentes y gestión de estado

La aplicación se ha construido sobre el *framework* Next.js 15, aprovechando su arquitectura basada en *App Router*. Esta elección ha permitido implementar una estrategia híbrida de renderizado: las estructuras estáticas y de navegación sirven como *Server Components* para optimizar la carga inicial, mientras que los paneles interactivos (*grid*, formularios) operan como *Client Components*.

Para la gestión del estado global, se ha descartado el uso de contextos nativos de React en favor de la librería Zustand. Esta decisión responde a la necesidad de gestionar estados complejos y frecuentemente mutables (como la lista de sesiones cargadas o el progreso de subida de múltiples archivos) sin provocar re-renderizados innecesarios en toda la jerarquía de componentes. El almacén principal (*store*) actúa como una máquina de estados finitos, controlando las transiciones de los archivos desde su puesta en cola (*pending*), pasando por el análisis (*uploading*), hasta su confirmación (*done*).

En cuanto a la comunicación con el servidor, se ha optado por una implementación ligera basada en *fetch* nativo y *hooks* de efecto (*useEffect*). Dado que la naturaleza de los datos es efímera (el estado de una sesión de edición no necesita caché persistente entre recargas), esta aproximación reduce la complejidad y el tamaño del *bundle* final frente a soluciones más pesadas de gestión de caché asíncrona.

5.4.2. Implementación de la página principal y tableros

La estructura de navegación se organiza mediante un sistema de *layouts* anidados que proporciona coherencia visual a través de toda la aplicación. La interfaz implementa una barra lateral persistente (*sidebar*) que segrega las funcionalidades en cuatro áreas lógicas: “Planificación”, “Subidas”, “Datos” y “Panel de Control”. Este componente se mantiene visible en todas las vistas de gestión interna, permitiendo al usuario conmutar entre el editor de horarios y las herramientas de ingesta sin perder el contexto de su sesión.

La vista principal de aterrizaje, denominada “Panel de Control”, actúa como un distribuidor central de tareas. A diferencia de un tablero analítico tradicional, se ha diseñado como un *hub* de accesos directos organizado en tarjetas funcionales que cubren tres áreas clave: gestión de archivos, agrupando los puntos de entrada para la ingesta de fichas y horarios; planificación y conflictos, que ofrece acceso inmediato al editor de rejilla y al panel de auditoría; y visualización de datos, para la consulta rápida de los catálogos maestros (profesores, aulas, asignaturas) en modo lectura.

Esta arquitectura de información reduce la carga cognitiva del gestor académico, presentándole únicamente las acciones relevantes agrupadas por intención de uso, en lugar de exponer toda la complejidad del sistema en un solo menú jerárquico.

5.4.3. Implementación de las páginas de ingesta

El módulo de carga de archivos destaca por una implementación nativa del estándar *drag and drop*. En lugar de depender de librerías externas que aumentarían el peso de la aplicación,

se ha desarrollado un componente **Dropzone** propio que interactúa directamente con los eventos del navegador (**dragover**, **drop**). Esto permite interceptar los archivos PDF directamente del sistema operativo, validando su tipo MIME (*multipurpose internet mail extensions*) en el cliente antes de iniciar cualquier transferencia.

Un desafío técnico en esta fase es la latencia inherente al procesamiento de PDFs en el servidor. Para mitigar la incertidumbre del usuario durante esta espera, se ha implementado un sistema de *feedback* reactivo inmediato. Al iniciar la extracción, la interfaz sustituye los controles de acción por indicadores de carga circulares (*spinners*) acompañados de etiquetas de estado dinámicas (“Procesando” y “Completado”), confirmando visualmente que el proceso en segundo plano sigue activo y evitando la percepción de bloqueo de la aplicación mientras el *backend* ejecuta la extracción síncrona.

5.4.4. Componente *grid* interactivo

El núcleo visual del proyecto es el componente de rejilla horaria, diseñado para permitir la manipulación directa de los horarios. Su implementación combina técnicas de maquetación avanzada y APIs de bajo nivel:

- **Arquitectura visual:** la estructura base utiliza **CSS Grid** para definir las columnas de los días. Sin embargo, el posicionamiento interno de los bloques es absoluto y se gestiona mediante un algoritmo de disposición inteligente. Para resolver la coexistencia visual de sesiones solapadas, se ha implementado un sistema de “carriles”: cuando se detecta una coincidencia temporal, el componente divide dinámicamente el ancho de la columna, calculando el desplazamiento horizontal (**left**) y la anchura (**width**) necesaria para que ambas sesiones se muestren en paralelo sin ocultarse mutuamente. Simultáneamente, la posición vertical y la altura se derivan matemáticamente de la hora de inicio y duración, garantizando una representación fidedigna de la escala temporal.
- **Motor *drag & drop* nativo:** al igual que en la ingesta, la interactividad de “arrastrar y soltar” sesiones para cambiar su horario se ha construido sobre la **HTML5 Drag and Drop API**. Al iniciar el arrastre, el componente genera una imagen fantasma (*ghost element*) semitransparente. Al soltar el elemento, el sistema captura las coordenadas (*X, Y*) del ratón, calcula sobre qué franja horaria se ha realizado el evento y actualiza el estado local, disparando instantáneamente una re-evaluación de conflictos sin necesidad de recargar la página.

5.4.5. Integración visual de conflictos

La visualización de errores no es un proceso pasivo, sino que se integra directamente en los componentes de la interfaz. El sistema de renderizado de las tarjetas de sesión evalúa en tiempo real la bandera de conflicto (**hasConflict**) inyectada por el *backend*.

Cuando esta bandera es positiva, el componente aplica dinámicamente estilos de alerta utilizando clases de utilidad de Tailwind CSS (bordes rojos, sombras difusas y coloración de fondo específica).

Complementariamente, la gestión detallada de incidencias se centraliza en el “Panel de Auditoría”, cuya interfaz se muestra en la Figura 4.6. Este componente tabular no se limita a alertar, sino que desglosa la anatomía del conflicto presentando explícitamente las dos sesiones involucradas en la colisión (asignatura, grupo y ubicación temporal). Esta visualización comparativa facilita la toma de decisiones mediante una lógica de resolución inteligente: si el conflicto es local, el sistema permite navegar directamente a la celda afectada; si es complejo (entre diferentes titulaciones), despliega un diálogo modal para la desambiguación manual sin perder el contexto de ninguna de las partes.

Capítulo 6

Validación y pruebas

La fase de validación y pruebas tiene como objetivo fundamental garantizar que el sistema no solo cumple con los requisitos técnicos especificados, sino que también responde con precisión a las necesidades del usuario final en entornos de uso real. Para este proyecto, se ha adoptado un enfoque de validación pragmático centrado en la integridad de los algoritmos críticos.

Dada la naturaleza algorítmica del procesamiento de documentos y la gestión de conflictos, la estrategia de aseguramiento de la calidad se articuló en un modelo de validación dual. Este comprendió, por un lado, una verificación técnica de bajo nivel mediante *scripts* de ejecución directa para auditar la precisión del *backend* y, por otro, una validación funcional de sistema desde la interfaz de usuario para certificar la correcta integración de todos los componentes. El presente capítulo detalla los protocolos seguidos para asegurar la estabilidad, la fiabilidad de los datos y la experiencia de usuario, priorizando la detección de errores en los flujos de trabajo más críticos del sistema.

6.1. Validación de la lógica de negocio

La fiabilidad de este sistema depende de la integridad de los datos extraídos y de la precisión del motor de conflictos. Dado que el núcleo del proyecto reside en la interpretación de documentos no estructurados (PDF), se descartó un enfoque basado exclusivamente en aserciones binarias en favor de una metodología de validación empírica orientada a la calidad del dato. Mediante una estrategia de “inspección de tuberías de datos” (*data pipeline inspection*), cada etapa del procesamiento fue sometida a pruebas de estrés con *scripts* de control y *datasets* de referencia, permitiendo no solo verificar la ejecución del código, sino cuantificar la fidelidad con la que el sistema interpreta la realidad académica de los archivos originales.

No obstante, esta flexibilidad en la fase de extracción se complementó con un rigor estricto en los componentes donde la lógica de negocio es determinista. Para el motor de detección de conflictos y los módulos de dominio, se implementaron suites de pruebas basadas en aserciones tradicionales (*asserts*). Mediante el uso de `pytest` y la generación de datos sintéticos controlados, se verificó de forma binaria que las reglas de solapamiento y las restricciones de integridad de los modelos (CRUD) se cumplieran sin margen de error.

6.1.1. Verificación de los procesos de extracción y *parsing*

Dada la naturaleza de las fuentes de datos (fichas académicas y horarios en PDF), la validación de esta capa no se limitó a una verificación de ejecución, sino a un análisis de la fidelidad en la transformación del dato. Para ello, se empleó un protocolo de inspección de resultados basado en *datasets* de control.

Se seleccionó un corpus de documentos reales representativos de la casuística académica de la titulación, incluyendo:

- **Fichas académicas:** archivos como `G49.pdf` y `G652.pdf`, utilizados para validar la extracción de metadatos, competencias y descriptores.
- **Horarios académicos:** versiones complejas de documentos PDF que presentan desafíos de *layout* como celdas combinadas y múltiples sesiones simultáneas.

En lugar de aserciones binarias, el sistema de pruebas ejecutado mediante *scripts ad-hoc* (`test_extractor.py` y `test_parsing.py`) generó métricas de diagnóstico integradas en el flujo de salida. El motor de extracción implementó una función de evaluación de calidad (*scoring*), permitiendo categorizar los resultados según el nivel de confianza:

- **Confidence score:** media del 83.78 % en la extracción de texto desde PDFs nativos, garantizando una base sólida para el posterior *parsing*.
- **Extraction quality:** clasificación automática de los resultados en niveles (*good*, *acceptable*, *unusable*), lo que permite al sistema identificar proactivamente cuándo un documento requiere intervención manual.

Cada ejecución de prueba generó artefactos en formato JSON y TXT persistidos en la estructura de carpetas `results/`. Estos archivos permitieron realizar una auditoría técnica de los resultados del *parser*, verificando que campos críticos como el `codigo_plan`, `nombre_asignatura` y los horarios de las sesiones docentes se mapearan correctamente a las entidades de dominio sin pérdida de información durante la normalización.

Los *scripts* de prueba fueron diseñados para detectar y reportar *warnings* específicos en lugar de interrumpir el flujo (por ejemplo, nombres de aula fragmentados o códigos de asignatura incompletos). Esta estrategia de validación permitió iterar sobre las expresiones regulares y los algoritmos de detección de tablas hasta alcanzar una tasa de éxito operativa superior al 95 % en documentos con texto embebido.

6.1.2. Validación del motor de conflictos

A diferencia de la capa de extracción, donde predominó la validación empírica, el motor de detección de conflictos se validó mediante un enfoque de pruebas unitarias estrictas. Al tratarse de un componente determinista basado en reglas lógicas, se emplearon aserciones binarias (*asserts*) bajo el *framework* `pytest` para garantizar que cada restricción académica se cumpliera sin ambigüedades.

Para evitar la dependencia de la base de datos en esta fase, se utilizaron funciones auxiliares de generación de datos sintéticos (*mock data*). Esto permitió crear sesiones docentes controladas para evaluar escenarios límite que serían difíciles de replicar con datos reales. Los principales casos de uso validados incluyeron:

- **Solapamiento de aula:** verificación de que dos sesiones en la misma ubicación y franja horaria disparen un conflicto de tipo `SOLAPAMIENTO_AULA` con severidad crítica.
- **Solapamiento de profesor:** validación de la lógica de detección para docentes asignados simultáneamente a grupos distintos, distinguiendo correctamente entre docencia única (conflicto crítico) y situaciones de co-docencia permitidas.
- **Conflictos de grupo:** comprobación de la integridad de los horarios para un mismo grupo de alumnos, asegurando la detección de solapamientos en sesiones de teoría y la correcta validación de desdobles en prácticas de laboratorio.
- **Restricciones de conciliación:** validación de las reglas de negocio que impiden la asignación de sesiones fuera de los rangos horarios permitidos para profesores con restricciones específicas.

Las pruebas ejecutadas en `test_engine_rules.py` permitieron medir la eficiencia de los algoritmos de detección. Los resultados demostraron una alta velocidad de procesamiento, con tiempos de respuesta inferiores a 0,01s por validación de sesión.

La ejecución de la suite completa de `pytest` arrojó una tasa de éxito del 100% en las reglas de negocio implementadas. La arquitectura de estas pruebas permitió realizar pruebas de regresión inmediatas: cada vez que se ajustaba un parámetro en el motor, se volvían a ejecutar las aserciones para asegurar que los cambios no introducían errores en reglas previamente validadas.

6.1.3. Pruebas de integridad de dominio y persistencia

La última capa de validación del *backend* se centró en la robustez de la infraestructura base y la integridad de los modelos de dominio. El objetivo de estas pruebas fue garantizar que cualquier dato procesado por los extractores o manipulado por el usuario a través del sistema CRUD cumpliera con las restricciones estructurales y de tipo definidas en el diseño.

Se utilizaron los modelos de `Pydantic` para realizar una validación de datos en tiempo de ejecución. Se verificaron los siguientes aspectos:

- **Restricciones de valor:** aserciones para rechazar datos inconsistentes, como créditos negativos o nombres de asignatura vacíos, mediante la captura de excepciones `ValidationError`.
- **Normalización de *enums*:** validación de que los campos como `PeriodoAcademico` o `TipoAsignatura` solo acepten valores predefinidos, evitando la degradación de la integridad de los datos en la base de datos.

Las pruebas de integración en los módulos de recursos y docencia aseguraron la correcta comunicación con la base de datos. Se ejecutaron ciclos completos de vida de la información (*create, read, update, delete*) para entidades críticas como `Aula`, `Profesor` y `SesionDocente`.

- **Estabilidad de la conexión:** uso de `test_database.py` para confirmar la gestión de sesiones de base de datos y la correcta ejecución de migraciones.
- **Persistencia de relaciones:** verificación de que los identificadores únicos y las claves foráneas se mantienen consistentes tras operaciones de actualización complejas.

Finalmente, se validó la configuración del sistema mediante `test_settings.py`, asegurando que las variables de entorno, rutas de almacenamiento de resultados y parámetros de conexión se cargaran correctamente en los distintos entornos de ejecución. Este conjunto de pruebas garantiza que, una vez superada la fase de extracción, el sistema opera sobre una base de datos estable y con información plenamente normalizada, minimizando el riesgo de errores en tiempo de ejecución durante la interacción con la interfaz de usuario.

6.2. Validación del sistema e integración funcional

Una vez verificada la robustez de los algoritmos y la integridad de la base de datos en el *backend*, se procedió a la fase de pruebas de sistema. En esta etapa, el enfoque se desplazó hacia una metodología de “caja negra”, donde la validación se realizó exclusivamente a través de la UI, simulando el flujo de trabajo real de un gestor académico.

6.2.1. Metodología de pruebas de caja negra

El objetivo principal fue confirmar que los componentes individuales, una vez integrados, cumplieran con los requisitos funcionales del sistema. Se definieron escenarios de prueba “extremo

a extremo” (*end-to-end*), monitorizando la respuesta del sistema ante entradas manuales y cargas de archivos desde el navegador.

Se validó el ciclo de vida de la información desde su origen físico hasta su persistencia:

1. **Carga de documentos:** selección y subida de archivos PDF a través del módulo de importación.
2. **Feedback de procesamiento:** verificación de que el sistema muestra estados de carga y mensajes de éxito/error tras la ejecución de los *pipelines* de extracción.
3. **Visualización *post-parsing*:** confirmación de que los datos extraídos (asignaturas, grupos, sesiones) se presentan correctamente en las tablas y formularios de la interfaz sin errores de codificación o formato.

6.2.2. Validación de la interacción y gestión de conflictos

Uno de los puntos críticos de la validación fue la respuesta dinámica de la interfaz ante la edición de horarios y la detección de colisiones en tiempo real. Se ejecutaron pruebas de interacción para asegurar que cualquier modificación manual (cambio de aula, ajuste de hora o asignación de profesor) desencadenara las siguientes acciones:

- Llamada inmediata a las API del *backend* para la re-evaluación de reglas.
- Actualización del estado de la interfaz sin necesidad de recargar la página completa.
- Persistencia correcta de los cambios en la base de datos tras la confirmación del usuario.

Se verificó que el sistema de notificaciones de conflictos fuera funcional y comprensible. Se forzaron errores de planificación *ad-hoc* (por ejemplo, asignar dos clases al mismo profesor a la misma hora) para validar que:

- Los conflictos aparecen destacados con el código de color y severidad correspondiente.
- La descripción del error es clara y permite al usuario identificar qué regla se ha vulnerado.

6.2.3. Estabilidad y usabilidad del sistema

Finalmente, se realizaron pruebas de navegación para asegurar una experiencia de usuario fluida. Se validó la respuesta del sistema ante acciones comunes como el filtrado de horarios por titulación, la búsqueda de asignaturas y el manejo de excepciones (por ejemplo, intentar cargar un archivo con formato no soportado), confirmando que el sistema guía al usuario mediante mensajes de error controlados en lugar de producir fallos inesperados en la aplicación.

Este proceso de validación manual desde la UI sirvió como la última línea de defensa, garantizando que la potencia del motor de análisis del *backend* se traduce en una herramienta útil, intuitiva y fiable para la gestión académica.

6.3. Pruebas de aceptación

Como fase final del proceso de validación, se realizaron pruebas de aceptación con el usuario objetivo del sistema, la Jefa de Estudios de la Facultad de Ciencias. Durante esta sesión, se verificó la utilidad de la herramienta en escenarios de gestión real, obteniendo una valoración positiva en cuanto a la precisión de los resultados y la optimización de tiempos. La usuaria propuso la incorporación de una funcionalidad adicional para mejorar la flexibilidad operativa, la cual han sido implementadas con éxito. No obstante, dado que la redacción de la presente memoria se encontraba en una fase avanzada, el detalle técnico de su diseño e implementación no se incluye en los capítulos anteriores, quedando documentadas junto a otras propuestas de mejora en la sección de líneas de trabajo futuro del siguiente capítulo.

Capítulo 7

Conclusiones y trabajo futuro

La culminación de este TFG representa la transición exitosa de un modelo de gestión académica tradicional, dependiente de procesos manuales y hojas de cálculo genéricas, hacia una plataforma *full-stack* reactiva e integral. El desarrollo ha demostrado que es posible sustituir la validación humana por una validación algorítmica sistemática, eliminando inconsistencias críticas como la sobreocupación de aulas o la bilocación docente. La arquitectura diseñada bajo el modelo C4 y el estilo REST ha garantizado un sistema desacoplado, escalable y capaz de mantener la integridad transaccional en todo momento.

Grado de cumplimiento de los objetivos

Tras contrastar los resultados obtenidos con los hitos definidos en la fase de planificación, se confirma el cumplimiento satisfactorio de los siguientes objetivos:

- **Ingeniería de datos y normalización:** se ha cumplido el objetivo de transformar información no estructurada de fichas docentes y horarios en un modelo relacional coherente mediante técnicas de *parsing* nativo y el motor *Gridmaster*.
- **Motor de detección de conflictos:** se ha construido una lógica de negocio central capaz de identificar automáticamente violaciones de integridad espacial, temporal y de conciliación docente, clasificándolas por niveles de severidad.
- **Herramientas de edición y UX (*user experience*):** se ha implementado una interfaz de usuario avanzada que permite la gestión CRUD y la edición interactiva mediante *drag and drop*, ofreciendo una alternativa robusta a las herramientas ofimáticas genéricas.
- **Optimización mediante coincidencia difusa:** objetivo cumplido mediante la resolución automática de discrepancias tipográficas en la ingesta. La efectividad del *fuzzy matching* ha demostrado ser dependiente de la legibilidad del documento fuente, eliminando la intervención manual en escenarios estándar y limitándola a casos de degradación severa del PDF.
- **Garantía de persistencia ACID:** se ha implementado una arquitectura de base de datos que asegura la atomicidad y consistencia de las operaciones masivas de edición, protegiendo la integridad de la planificación ante fallos.

Análisis de impacto

El proyecto trasciende la mera digitalización, aportando un valor significativo en términos de sostenibilidad y calidad educativa:

- **Optimización de recursos:** la automatización de la ingesta reduce drásticamente el tiempo de dedicación del personal administrativo, permitiendo una reasignación de esfuerzos hacia tareas de mayor valor añadido.
- **Calidad del servicio educativo:** al minimizar los errores de planificación que derivan en interrupciones lectivas, el sistema garantiza un entorno académico estable, mejorando directamente la experiencia de alumnos y profesores.
- **Consistencia social:** el módulo de conciliación respeta las restricciones horarias particulares del profesorado, promoviendo una gestión más humana del calendario académico.

Líneas de trabajo futuro

Para solventar las limitaciones identificadas durante el desarrollo y expandir el alcance funcional del sistema diseñado, se proponen las siguientes líneas de evolución técnica:

- **Implementación de un sistema multiusuario y gestión de roles:** dado que el sistema actual se concibe como una herramienta de administración de acceso único, una evolución crítica es integrar un módulo de autenticación y control de acceso basado en roles. Esto permitiría segmentar las capacidades de edición de los gestores académicos frente a perfiles de solo consulta para el profesorado y alumnado.
- **Mejora de la robustez en la ingesta mediante OCR avanzado:** para mitigar la dependencia de la legibilidad estructural de los ficheros PDF originales, se propone integrar motores de reconocimiento óptico de caracteres basados en inteligencia artificial. Esta mejora permitiría procesar con éxito documentos escaneados o con variaciones estructurales severas que actualmente requieren intervención manual.
- **Plataforma de consulta personalizada y sincronización externa:** como extensión a la gestión de sesiones, se plantea el desarrollo de una interfaz de visualización mediante componentes de calendario interactivo compatibles con estándares como Google Calendar o iCalendar. El objetivo es que cualquier ajuste validado se sincronice automáticamente en los dispositivos personales de los usuarios afectados, eliminando la brecha comunicativa entre la gestión y la ejecución docente.
- **Generación automática de horarios mediante inteligencia artificial (IA):** evolucionar el motor de validación actual, que actúa como una herramienta de auditoría y notificación, hacia un sistema de planificación proactiva. Mediante algoritmos de optimización combinatoria o modelos de IA Generativa, el sistema podría proponer horarios desde cero que respeten todas las restricciones de recursos físicos, humanos y de conciliación.
- **Integración con sistemas ERP institucionales:** con el fin de evitar que la herramienta opere como un silo de información, se propone desarrollar una API de comunicación directa con los sistemas de gestión universitaria existentes. Esto permitiría el volcado automático de la planificación validada directamente en las bases de datos oficiales, eliminando procesos de exportación de documentos intermedios.
- **Gestión automatizada de restricciones docentes mediante archivos externos:** a partir del *feedback* obtenido en las pruebas de aceptación, se propone la creación de un motor de ingesta capaz de procesar archivos de hojas de cálculo que contengan la totalidad de las restricciones de imposibilidad del profesorado. Esta mejora permitiría centralizar y automatizar la validación de todas las limitaciones del personal (incluyendo la conciliación previamente detallada y otras restricciones administrativas), transformando el motor de conflictos en una herramienta capaz de identificar instantáneamente cualquier vulneración de la disponibilidad docente durante el proceso de planificación.

Bibliografía

- [1] Pressman, R. S. (2010). *Software Engineering: A Practitioner's Approach*. McGraw-Hill.
- [2] Cohn, M. (2004). *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional.
- [3] Sommerville, I. (2011). *Software Engineering*. Addison-Wesley.
- [4] Haerder, T., & Reuter, A. (1983). *Principles of transaction-oriented database recovery*. ACM Computing Surveys (CSUR), 15(4), 287-317.
- [5] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.
- [6] Levenshtein, V. I. (1966). *Binary codes capable of correcting deletions, insertions, and reversals*. Soviet Physics Doklady, 10(8), 707-710.

Apéndice A

Ejemplo de ficha docente

Se adjunta a continuación, a modo de muestra representativa, la guía docente oficial de la asignatura *Natural Language Processing* (G687).

Este documento constituye la prueba material de la estructura semi-estructurada que el sistema debe interpretar. Su inclusión en este anexo permite visualizar la disposición original de los datos (tales como el código de la asignatura, la carga de créditos ECTS, la coordinación docente y la adscripción a la titulación) en su formato nativo, evidenciando la dispersión de la información que el sistema normaliza para construir el catálogo académico.

DATOS DEL TÍTULO		
DENOMINACIÓN DEL TÍTULO	Tipo	Curso
Grado en Ingeniería Informática	OPTATIVA	4

DATOS DE LA ASIGNATURA						
CENTRO RESPONSABLE :	32	FACULTAD DE CIENCIAS				
CODIGO Y DENOMINACION :	G687	NATURAL LANGUAGE PROCESSING				
DEPARTAMENTO RESPONSABLE :	21	DPTO. MATEMATICAS, ESTADISTICA Y COMPUTACION				
IMPARTICIÓN:	Presencial	N° CRÉDITOS ECTS:	6,00	SECUENCIA:	Cuatrimestral	N°: 2 ALUMNOS: 0

MODALIDADES ORGANIZATIVAS Y MÉTODOS DOCENTES																
	PRESENCIALES										NO PRESENCIALES					TOTAL
	CLASES						SEGUIMIENTO			TOTAL	TG	TA	TU-NP	EV-NP	TOTAL	
	TE	PA	PLE	PLO	CL	TOTAL	TU	EV	TOTAL							
HORAS PROGRAMA	20,00	10,00	0,00	30,00	0,00	60,00	7,50	7,50	15,00	75,00	30,00	45,00	0,00	0,00	75,00	150,00
HORAS IMPARTIDAS	20,00	10,00	0,00	30,00	0,00	60,00										

ASIGNATURA	G687	Natural Language Processing
-------------------	------	-----------------------------

DATOS DEL PROFESORADO										
TIPO PROFESOR	APELLIDOS Y NOMBRE	UNIVERSIDAD	R	TE	PA	PLE	PLO	CL	TOTAL	% SEGU.
CD	TIRNAUCA , CRISTINA	Universidad de Cantabria	S	20,00	10,00	0,00	30,00	0,00	60,00	100,00
TOTAL									60,00	100,00

DESGLOSE DE LA ASIGNATURA					
TIPO	PARTE / PRÁCTICA	HORAS	Nº GRUPOS	Nº PROFESORES X GRUPO	HORAS IMPARTIDAS
T	T01	20,00	1	1	20,00
O	O01	30,00	1	1	30,00
A	A01	10,00	1	1	10,00
TOTALES:		60,00			60,00

Fecha de acuerdo del Consejo de Departamento:	14-05-2025	Fecha de acuerdo de la Junta de Centro:	
--	------------	--	--

Vº Bº El/La Decano/a – Director/a del centro	
Fdo:	

Apéndice B

Ejemplo de cuadro horario

Se adjunta a continuación la planificación horaria oficial correspondiente al primer cuatrimestre del grado en matemáticas, empleada para la validación del módulo de ingesta de horarios.

La inclusión de este documento permite visualizar el desafío de la reconstrucción espacial al que se enfrenta el sistema. A diferencia de un texto lineal, este formato evidencia una estructura matricial donde la información crítica, como la duración de una sesión, el día de la semana o el solapamiento de grupos, no reside explícitamente en el texto de cada celda, sino que debe inferirse a partir de sus coordenadas geométricas y su posición relativa dentro de la rejilla del calendario.

PRIMER CUATRIMESTRE

	PRIMER CURSO								
	LUNES		MARTES		MIÉRCOLES		JUEVES		VIERNES
8:30 9:30	ODS AULA 2		Sistemas Digitales AULA 2		Algebra Lineal y Discreta AULA 2				
9:30 10:30			Algebra Lineal y Discreta AULA 2		Fundamentos Físicos AULA 2				
10:30 11:30	Fund. Físicos Grupo PL3 AULA 13 (*)	Sistemas Digitales PL1 y PL2 SEM.INF	Fund. Físicos PL2 AULA 13 (*)	Intro al software PL3 y PL4 LSC 1	Intro al software TE AULA 2		Fundamentos Físicos AULA 14		Sist. Digitales TE / PA 1 AULA 14
11:30 12:30					Sistemas Digitales PL4 SEM.INF		ODS AULA 14		Algebra Lineal y Discreta AULA 14
12:30 13:30	Fund. Físicos Grupo PL1 AULA 13 (*)	Sistemas Digitales PL3 SEM.INF	Fund. Físicos Grupo PL4 AULA 13 (*)	Introduc. al software PL1 y PL2 LSC 1					
13:30 14:30							Intro al software PA3 AULA 5		Intro al software PA4 AULA 7
14:30 15:30									
15:30 16:30									
16:30 17:30									
17:30 18:30									
18:30 19:30									

(*) La programación de prácticas de laboratorio y simulación se dará a conocer a principio de curso.

Horas reservadas para clases de recuperación, coordinadas con el responsable de la titulación

	SEGUNDO CURSO									
	LUNES			MARTES		MIÉRCOLES		JUEVES		VIERNES
8:30 9:30										
9:30 10:30										
10:30 11:30								Estructura de Computadores PL4 LATC		
11:30 12:00				Estructura de Computadores PL3 LATC						
12:00 12:30										
12:30 13:30										
13:30 14:30										
14:30 15:30		Estructura Computadores PA1 AULA 2		Estructura de Computadores TE/tutoría AULA 2		Estructura de Computadores AULA 2		Estructur a de Datos PA1 AULA 9	Estruct Computa dores PA2 AULA 2	Int.persona-comp. PL1 y PL2 AULA 7
15:30 16:30	Matemática Discreta AULA 2			Interacción persona-computador TE AULA 7		Matemática Discreta AULA 2		Estructuras de Datos TE AULA 2		
16:30 17:30				Interacción persona-comp. TE / PA1 y PA2 AULA 7				Estructuras de Datos TE / PA 2 AULA 2		Estructura de Computadores PL5 LATC
17:30 19:30	Estructur as de Datos PL2 LSC 1	Estructura de Computadores PL1 L ATC		Estructuras de Datos PL1 LSC 1	Estructur a de Computa dores PL2 LATC	Econ. y ADE AULA 2		Econ. y ADE AULA 2		
										AULA 2

Horas reservadas para clases de recuperación, coordinadas con el responsable de la titulación

TERCER CURSO									
	LUNES		MARTES	MIÉRCOLES		JUEVES		VIERNES	
08:30 09:30				Int. Persona-comp. PL1 y PL2 AULA 14		Prog. Paralela, Concurrente y de Tiempo Real PL3 LTR/ATC		Prog. Paralela, Concurrente y de Tiempo Real PL5 LTR/ATC	Introducción a los Sistemas Inteligentes PL3 AULA 13
09:30 10:30	Programación Paralela, Concurrente y de Tiempo Real AULA 14	Programación Paralela, Concurrente y de Tiempo Real AULA 14							
10:30 11:30	Interacción Persona-computador TE AULA 14	Introducción a los Sistemas Inteligentes AULA 14	Introducción a los Sistemas Inteligentes AULA 14	Intr. a Redes de Computadoras TE AULA 14	Intr. a Redes de Computadoras PA2 AULA 13	Intr. a Redes de Computadoras PA1 AULA 13	Ingeniería de Software I PA 2 SEM INF	Ingeniería de Software I PL3 SEM INF	Introducción a los Sistemas Inteligentes PL2 AULA 13
11:30 12:30	Ingeniería de Software I AULA 14	Intr. a Redes de Computadoras TE AULA 14				Redes de Computadoras PA1 SEM INF			
12:30 13:30	Prog. Paralela, Concurrente y de Tiempo Real PL1 LTR/ATC	Interacción Persona-comp. TE / PA1 y PA2 AULA 14	Prog. Paralela, Concurrente y de Tiempo Real PL2 LTR/ATC		Prog. Paralela, Concurrente y de Tiempo Real PL4 LRT	Ingeniería de Software I PL1 SEM INF	Introducción a Redes de Computadoras PL1 LATC	Ingeniería de Software I PL2 SEM INF	Introducción a los Sistemas Inteligentes PL1 AULA 13
13:30 14:30		AULA 14							
14:30 15:30									
15:30 17:30			Introducción a Redes de Computadoras PL2 LATC			Introducción a Redes de Computadoras PL3 LATC			
17:30 19:30						Introducción a Redes de Computadoras PL4 LATC			

 Horas reservadas para clases de recuperación, coordinadas con el responsable de la titulación

	CUARTO CURSO MENCIÓN EN COMPUTADORES								
	LUNES	MARTES			MIÉRCOLES	JUEVES			VIERNES
08:30 09:30									
09:30 10:30									
10:30 11:30									
11:30 12:30									
12:30 13:30									
13:30 14:30									
14:30 15:30	Diseño y Administración de Redes AULA 15	Computer Animation and Videogame Portátiles AULA 13	Bases de Datos Avanza AULA 15	Sist. Informa Empresa AULA 5	Sistemas Embebidos AULA 15	Computer Animation and Videogame Portátiles AULA 13	Bases de Datos Avanza AULA 15	Sist. Informa Empresa AULA 5	Sistemas Embebidos AULA LSC 3
15:30 16:30	Multiprocesadores AULA 15								
16:30 17:30		Sistemas Operativos Avanzados UNICAN-LABS AULA 15	Multiprocesadores LSC 3			Diseño y Administración de Redes LSC DICOM - AULA 15			
17:30 18:30	Sistemas Operativos Avanzados UNICAN-LABS AULA 15								
18:30 19:30			AULA 15			Diseño y Administración de Redes AULA 15			

Horas reservadas para clases de recuperación, coordinadas con el responsable de la titulación

CUARTO CURSO MENCIÓN EN COMPUTACIÓN											
	LUNES	MARTES			MIÉRCOLES		JUEVES		VIERNES		
08:30 09:30											
09:30 10:30											
10:30 11:30											
11:30 12:30											
12:30 13:30											
13:30 14:30											
14:30 15:30	Modelos de Cálculo AULA 13	Computer Animation and Videogame Portátiles AULA 13	Bases de Datos Avanzadas AULA 15	Sist. Informa Empresa AULA 5	Representación del Conocimiento AULA 13	Computer Animation and Videogame Portátiles AULA 13	Bases de Datos Avanzada AULA 15	Sist. Informa Empresa AULA 5	Representación del Conocimiento AULA 13		
15:30 16:30											
16:30 17:30	Gráficos por Computador y Realidad Virtual AULA 13	Diseño de Algoritmos AULA 13			Modelos de Cálculo AULA 13	Gráficos por Computador y Realidad Virtual AULA 13			Diseño de Algoritmos AULA 13		
17:30 18:30											
18:30 19:30						AULA 13					

Horas reservadas para clases de recuperación, coordinadas con el responsable de la titulación

CUARTO CURSO MENCIÓN EN I. SOFTWARE									
	LUNES		MARTES		MIÉRCOLES		JUEVES		VIERNES
08:30 09:30									
09:30 10:45									
10:30 11:30									
11:30 12:30									
12:30 13:30									
13:30 14:30									
14:30 15:30	Métodos de Desarrollo Software SEM INF	Computer Animation and Videogam Portátiles AULA 13	Bases de Datos Avanzadas AULA 15	Sist. Informa Empresa AULA 5	Procesos software SEM INF	Computer Animation and Videogame Portátiles AULA 13	Bases de Datos Avanzada AULA 15	Sist. Informa Empresa AULA 5	Métodos de Desarrollo Software SEM INF
15:30 16:30									
16:30 17:30	Calidad y Auditoría SEM INF	Diseño Software SEM INF			Calidad y Auditoría SEM INF	Procesos Software SEM INF			Diseño Software SEM INF
17:30 18:30									
18:30 19:30					SEM INF	Comp.Animation/Bases de Datos Adv./Sist Info Empresa AULA 13			

Horas reservadas para clases de recuperación, coordinadas con el responsable de la titulación